

N036 뉴스레터 에이전트 만들기_학 습서

Chapter01

뉴스레터 에이전트 — 수집부터 발행까지

만들 것

AI 업계에서는 매일 새로운 기술과 소식이 끊임없이 쏟아지고 있습니다. 최신 동향을 파악하는 것은 매우 중요하지만, 이를 매번 찾아보는 데는 많은 시간과 노력이 듭니다.

최신 뉴스를 자동으로 수집하고 핵심 정보만 선별해 알려주는 AI 에이전트가 있다면 어떨까요? 이번 시간에서는 정보 수집을 효율적으로 자동화해 줄 'AI 뉴스레터 에이전트' 제작 방법을 소개합니다.

소식을 수집하고 중요한 정보를 선별하는 데는 다양한 접근 방식이 있습니다. 그래서 이번 시간에서는 고려할 수 있는 여러 방법을 살펴보고, 그중 이 방식을 선택한 이유를 차근차근 소개해 드리겠습니다

다섯 단계로 나눠 보기

'뉴스레터 에이전트'라고 부르면 덩어리 하나처럼 들려서 어디부터 손대야 할지 막막합니다. 하지만 해야 할 일을 쪼개서 생각해보면 구현해야 하는 요소들이 단순해집니다.

1. 자료 수집. 어디서, 얼마나, 무엇을 믿고 가져올 것인가.

데이터를 가져오는 경로는 여러 개이고 각각 안정성·법적 위험·품질이 다릅니다. 데이터 수집 경로를 선정하는 기준에 대해서 고민해봅시다.

2. 중요도 선별. 수집된 여러 개의 정보를 모두 읽는 건 반쪽짜리 자동화겠죠. 그 중에서 '중요한 기사'를 선별하는 일은 쉽지 않은 일입니다. 어떻게 여러개의 기사를 3~5개로 추릴 수 있을 지 고민해봅시다.

3. 요약. 수집된 정보를 그대로 내보내기 보다는 의미있는 방식으로 요약해주면 독자가 훨씬 편리할 겁니다. 어떤 방식으로 요약할지, 어떤 관점으로 요약하게 할지도 고민할 지점입니다.

4. 검수. 요약을 진행했으면 AI가 제대로 요약을 했는지 검수하는 과정이 필요합니다. 요약본에 대한 검수는 어떻게 하면 좋을까요?

5. 발행. 선별하고 요약된 내용은 다른 사람이 볼 수 있게 발행하는 것까지가 의미있는 자동화가 되겠죠. 어떤 방식으로 발행하는 것이 좋을지 고민해봅시다.

섹션 배치

앞에서 설명한 다섯 단계가 섹션으로는 이렇게 배치됩니다.

단계	섹션	하는 일
준비	1~2	다섯 단계를 정하고, 빈 노드 다섯 개로 뼈대를 세운다
자료 수집	3~5	가져올 경로를 고르고, 소스를 재고, 수집 노드를 만든다
중요도 선별	6~7	고르는 구조를 정하고, 예선·본선을 만든다
요약	8~9	취재 노드를 만들고, 출력 스키마를 설계한다
검수	10	검수 노드를 만들고, 무엇과 대조할지 정한다
발행	11~12	Discord로 보내고, 매일 아침 자동으로 돌린다
운영	13~15	지표를 남기고, 내 분야로 옮기고, 정리한다

단계마다 먼저 무엇을 정해야 하는지 보고, 선택지를 늘어놓고, 하나를 골라 노드로 만들고, 전체를 실행해볼겁니다.

앞선 수업에서 다룬 Anthropic의 워크플로 패턴과 LangGraph를 활용해서 오늘의 자동화 워크플로를 구축해보겠습니다.

학습 목표

1. 뉴스레터 에이전트를 다섯 단계로 쪼개고, 그 단계를 그래프 노드로 세울 수 있습니다.
2. 자료를 가져오는 다섯 경로의 트레이드오프를 설명하고, 소스를 관문으로 검증할 수 있습니다.
3. 수집한 자료를 선별하는 파이프라인을 설계하고 검증할 수 있습니다.
4. 선별된 자료를 요약하고 검증하는 파이프라인을 설계하고 검수를 통과한 만큼만 발행할 수 있습니다.
5. Discord와 GitHub Actions로 매일 아침 실제로 받아 보게 만들고, 내린 결정을 퍼널 숫자로 검증할 수 있습니다.

결과물 미리 보기

아래는 이 파이프라인이 2026년 9월 4일 아침에 실제로 발행한 다섯 건입니다.
100건 남짓을 읽고 남긴 결과입니다.

오픈AI, 'GPT-6 아스트라' 출시..."AGI 시대 진입"	(전자신문)
오픈AI, 압도적 성능 'GPT-6 아스트라' 공개..."AGI 시대 진입" 선언	(AI타임스)
오픈AI, '아스트라'에 차세대 '순환 추론' 적용으로 성능 대폭 향상	(AI타임스)
Nvidia confirms it will buy Hugging Face for \$12.9 billion	(TechCrunch)
챗GPT·클로드·그록 등 주요 AI 서비스, 한때 일제히 '먹통'	(전자신문)

오늘 사용할 데이터

- **데이터셋:** [실시간 AI 뉴스 소스 (RSS · 공개 API)]
- **규모/구성:** TechCrunch·The Verge·OpenAI·DeepMind 블로그(RSS), AI타임스·전자신문(RSS, 한국어), Hacker News(공개 API) 등 11곳. 최근 24시간 기준 50~120건
- **라이선스:** 각 매체의 공개 RSS/API. 제목·요약과 원문 링크만 사용하고 본문 전문은 재배포하지 않습니다.
- **사용 범위:** 실시간 피드이므로 실행 시각마다 결과가 달라집니다.

주관식 퀴즈

Q. [아이스브레이킹] 여러분이 매일 챙겨 보는 분야의 소식이 있다면 무엇인가요? 지금은 어떤 경로로 그 소식을 접하고 있고, 그 방법에서 가장 번거로운 점 한 가지는 무엇인가요?

Chapter02

만드는 순서 정하기

섹션 1에서 다섯 단계를 정했습니다. 이제 하나씩 만들 차례인데, 그 전에 정할 게 하나 있어요. 무엇부터 만들 것인가요.

흔한 순서는 이렇습니다. 수집 함수를 만들고, 선별 함수를 만들고, 요약 함수를 만들고, 다 만든 뒤에 이어 붙입니다. 자연스러워 보이지만 끝까지 가기 전에는 도는 물건이 없고, 이어 붙이는 날에야 "이 함수가 돌려주는 것과 저 함수가 받는 게 다르네"를 발견하게 되죠.

우리는 반대로 갑니다. 다섯 단계를 빈 노드로 먼저 세우고 한 섹션에 하나씩 채웁니다. 앞선 수업에서 배운 LangGraph를 사용해서 노드를 구성하고 한 개씩 완성해 보겠습니다.

선택지

다섯 단계를 어떤 순서로 만들어 나갈 것인가의 선택지는 네 가지가 있고, 위에서 아래로 갈수록 미리 설계할 것이 줄어드는 대신 나중에 고칠 것이 늘어납니다.

① 부품을 다 만들고 마지막에 조립한다

수집·선별·요약·검수·발행 함수를 각각 완성한 뒤 마지막에 이어 붙입니다. 한 번에 한 가지만 생각하면 되고, 각 함수를 독립적으로 테스트하기 좋습니다. 대신 조립하는 날까지 도는 물건이 없습니다. 그리고 단계 사이의 규격 불일치가 가장 늦게, 한꺼번에 드러납니다.

② 빈 노드로 뼈대를 세우고 하나씩 채운다 ← 우리가 고른 것

다섯 노드를 로그만 남기는 껍데기로 만들어 먼저 연결하고, 섹션마다 하나씩 진짜 코드로 바꿉니다. 첫날부터 끝까지 도는 물건이 있습니다. 단계 사이의 규격(State 키)이 맨 처음에 정해지므로 나중에 어긋날 일이 없고, 어디까지 만들었는지가 실행 로그에 그대로 보입니다. 대신 State를 처음에 설계해야 합니다. 뒤에 무엇이 필요한지 아직 모르는 채로 키를 정하게 되고, 중간에 키가 늘면 앞 노드들을 다시 봐야 합니다.

③ 한 노드를 완성할 때마다 그래프를 다시 그린다

수집 노드를 완성하고 그래프를 만들고, 선별 노드를 만들면서 그래프를 고칩니다. 필요한 만큼만 State를 정의하므로 미리 설계할 것이 적습니다. 대신 그래프 정의를 매 섹션 다시 씁니다. 무엇이 바뀌었는지 학습자가 매번 대조해야 하고, 코드 중복도 늘어납니다.

④ 노트북 셀을 순서대로 실행하고 그래프는 쓰지 않는다

각 단계를 셀로 나눠 위에서 아래로 실행합니다. 가장 단순하고, 탐색 단계에서는 이것이 맞습니다. 대신 되돌아가기와 팬아웃을 표현할 수 없습니다. 그리고 매일 자동으로 도는 물건으로 옮길 때 전부 다시 써야 합니다.

우리는 '빈 노드로 뼈대를 세우고 하나씩 채운다' 쪽을 골랐습니다. 이 교안이 부품을 하나씩 붙여 나가는 방식이기 때문입니다. 섹션마다 노드를 하나 완성하고, 그때마다 전체를 한 번 돌려 봅니다. 그러면 "지금 어디까지 왔는가"가 실행 로그로 확인되고, 매 섹션의 결과가 다음 섹션의 입력이라는 것이 눈에 보입니다.

②의 약점인 'State를 미리 설계해야 한다'는 부담은 생각보다 작습니다. 섹션 1에서 다섯 단계를 정의할 때 각 단계가 무엇을 받아 무엇을 내놓는지를 이미 정했기 때문입니다. 수집은 기사 목록을 내놓고, 선별은 그중 다섯 건을 남기고, 취재는 요약을 붙입니다. State 키는 그 문장을 그대로 옮긴 것입니다.

조건이 달라지면 다른 선택지가 맞는 답이 됩니다. 무엇을 포기했는지 알고 고르면 됩니다.

State 정의하기

노드를 쓰기 전에 기사 하나를 어떤 모양으로 들고 다닐지부터 정해야 합니다. 소스마다 항목 이름이 다르고(`link` / `url` / `id`) 시각 형식도 제각각이거든요. 들어오는 걸 전부 하나의 그릇으로 바꿔 놓지 않으면 뒤의 모든 코드에 `if source == "hackernews": ...` 같은 분기가 생깁니다.

우리 그릇은 다섯 칸입니다 — `title` · `url` · `source` · `at` · `summary` .
여기에 뒤에서 채울 칸(`headline` · `summary` · `why`)이 붙고요.

State에는 단계 사이를 실제로 건너가는 것만 담습니다. 프롬프트 문자열이나 HTTP 세션처럼 한 노드 안에서만 쓰이는 건 올리지 않아요. 올리면 모든 노드가 그걸 들고 다니게 되고, 무엇이 어디서 쓰이는지 추적할 수 없게 됩니다.

먼저 실행해 보세요.

실행 되었습니다.

준비: LangGraph 설치

실행

1

실행 되었습니다.

실습: 다섯 단계를 빈 노드로 세우고 한 번 돌리기 (키 불필요)

실행

1

2

3

4

5

6

7

8

9

10

11

12

13

14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42

43

44

45

46

실제 실행 결과 — 빈 노드 다섯 개가 한 바퀴

- ① 수집 24시간 창 · 0건 (빈 노드)
- ② 선별 0 → 0건 (빈 노드)
- ③ 취재 0건 (빈 노드)
- ④ 검수 0건 (빈 노드)
- ⑤ 발행 0건 (빈 노드)

다섯 줄이 순서대로 찍혔습니다. 전부 0건이고 하는 일도 없지만 파이프라인은 완성되어 있어요.

앞으로 섹션마다 이 함수 중 하나를 진짜 코드로 바꿀 거고, 바꿀 때마다 숫자가 하나씩 0에서 벗어납니다.

세 가지만 짚어 둘게요.

첫째, 노드는 자기가 바꾼 키만 돌려줍니다. `verify` 와 `publish` 는 `log` 만 반환하죠. 언급하지 않은 키는 그대로 유지되니까, 나중에 State에 칸을 하나 더해도 기존 노드를 고칠 필요가 없습니다.

둘째, 리듀서는 두 곳에만 붙였습니다. `drafted` 는 섹션 9에서 여러 취재 워커가 동시에 쓰는 자리가 되고, `log` 는 모든 노드가 한 줄씩 남기는 자리입니다. 둘 다 합쳐야 하니 `operator.add` 가 필요해요. 반면 `collected` 와 `picked` 는 한 노드가 쓰고 다음 노드가 읽는 값이라 덮어쓰기가 맞습니다. 여기에 리듀서를 붙이면 선별이 9건을 5건으로 줄였는데 결과가 14건이 됩니다.

셋째, `build()` 가 노드를 `globals()[name]` 으로 찾습니다. 함수 객체를 직접 넘기지 않고 이름으로 찾으니까, 아래 섹션에서 `collect` 를 같은 이름으로 다시 정의하고 `build()` 를 다시 부르면 그것이 들어갑니다. 노드를 갈아 끼우는 데 고칠 코드가 없다는 뜻이에요.

넷째, State에는 단계 사이를 건너가는 것만 담았습니다. `hours` 는 수집 노드가 읽을 설정값이고 나머지 셋은 단계의 결과물이에요. 프롬프트 문자열이나 HTTP 세션처럼 한 노드 안에서만 쓰이는 건 올리지 않습니다. 올리면 모든 노드가 그걸 들고 다니게 되니까요.

걸어 다니는 뼈대

모든 단계를 아주 얇게 한 번씩 관통시켜 끝까지 도는 것을 먼저 만드는 방식입니다. 각 단계는 진짜 일을 하지 않고 다음으로 넘기기만 하죠. 방금 만든 빈 노드 다섯 개가 그겁니다.

이렇게 하면 규격이 어긋나는 걸 가장 일찍 발견합니다. 부품을 따로 만들면 수집이 돌려주는 모양과 선별이 기대하는 모양이 다르다는 사실을 조립하는 날 알게 되는데, 그날은 대개 마감 직전이에요. 뼈대를 먼저 세우면 그 규격이 첫날 State 선언 한 곳에 적히고 모든 노드가 그걸 보고 만들어집니다. 언제든 돌려 볼 수 있으니 "지금 어디까지 됐나"를 말로 설명할 필요도 없고요.

만드는 방법은 단순합니다. 각 단계를 입출력만 맞춘 껍데기(stub)로 만들어 전부 연결한 뒤, 한 번에 하나씩 진짜 구현으로 바꿉니다. 껍데기는 빈 값이나 고정된 가짜 값을 돌려주고 로그 한 줄을 남기면 됩니다. 바꿀 때마다 전체를 다시 돌려서 방금 바꾼 것 말고는 안 깨졌는지 확인하고요.

주의할 건 두 가지입니다. 뼈대가 너무 두꺼워지면 의미가 없어요. 껍데기에 조건 분기나 임시 로직을 넣기 시작하면 나중에 그걸 걷어내는 일이 또 생깁니다. 그리고 State를 처음에 완벽하게 설계하려 들지 마세요. 필요한 칸을 다 알 수 없는 게 정상이고 중간에 칸이 느는 것도 정상입니다. 미리 정해야 하는 건 칸의 목록이 아니라 누가 무엇을 넘겨주는지예요.

객관식 퀴즈

Q. `collected` 와 `picked` 에는 리듀서를 붙이지 않고 `drafted` 와 `log` 에만 붙였습니다. 가장 적절한 이유는?

- 1. 리스트 길이가 긴 키에만 리듀서가 필요하기 때문이다
- 2. `drafted` 는 여러 취재 워커가 동시에 쓰는 자리이고 `log` 는 모든 노드가 한 줄씩 남기는 자리라 합쳐야 하지만, 나머지는 한 노드가 쓰고 다음 노드가 읽으므로 덮어쓰기가 맞기 때문이다
- 3. `TypedDict` 는 키 두 개까지만 `Annotated` 를 허용하기 때문이다
- 4. 리듀서를 붙이면 실행 속도가 느려지므로 꼭 필요한 곳에만 붙인 것이다

• •

1. 리스트 길이가 긴 키에만 리듀서가 필요하기 때문이다

• •

2. `drafted` 는 여러 취재 워커가 동시에 쓰는 자리이고 `log` 는 모든 노드가 한 줄씩 남기는 자리라 합쳐야 하지만, 나머지는 한 노드가 쓰고 다음 노드가 읽으므로 덮어쓰기가 맞기 때문이다

• •

3. `TypedDict` 는 키 두 개까지만 `Annotated` 를 허용하기 때문이다

• •

4. 리듀서를 붙이면 실행 속도가 느려지므로 꼭 필요한 곳에만 붙인 것이다

주관식 퀴즈

Q. 껍데기 노드는 로그 한 줄만 남기고 빈 값을 돌려줍니다. "어차피 지울 코드인데 왜 만드나"라고 생각할 수 있습니다. 이 빈 노드들이 실제로 하는 일이 무엇인지 생각해 보세요.

Chapter03

자료 수집 — 첫 번째로 정할 것

다섯 단계 중 첫 번째입니다. 인터넷에 있는 글을 내 프로그램으로 가져와야 하는데 방법이 하나가 아니에요. 어느 경로를 쓰느냐가 이후의 안정성과 법적 위험을 거의 전부 정합니다.

선택지 — 다섯 칸의 사다리

정해야 할 것 — 인터넷에 있는 정보를 내 프로그램으로 가져오려면 어떤 경로를 써야 하는가?

아래 다섯 칸은 위로 갈수록 안전하고 아래로 갈수록 위험합니다. '사다리'라고 부르겠습니다. 위 칸이 가능하면 아래로 내려가지 않는 것이 원칙입니다.

① 공식 API

서비스가 "이렇게 가져가세요"라고 문서로 약속한 통로입니다. 인증 키를 받고 정해진 주소를 호출하면 구조화된 JSON이 돌아옵니다. 제공자가 API를 열어 두었다면 언제나 1순위입니다. 약속이 있으므로 형식이 바뀌면 공지가 오고, 막힐 걱정도 없습니다. 대신 열려 있는 곳이 드물고 한도가 걸립니다. 뉴스 분야에서는 이 칸이 거의 비어 있습니다.

② RSS · Atom 피드 ← 우리가 고른 것

언론사와 블로그가 "새 글이 올라오면 여기 적어 두겠다"고 발행하는 XML 파일입니다. 시간순으로 글이 쌓이는 정보에 가장 잘 맞습니다. 인증이 필요 없고 형식이 20년 넘게 안정적이며, 발행자가 스스로 공개한 것이라 의사가 분명합니다. 대신 제목과 짧은 요약만 주는 곳이 대부분이라 본문은 따로 가져와야 하고, 시각 표기 같은 세부가 소스마다 제멋대로라 섹션 5에서 따로 손봐야 합니다.

③ 공개 데이터셋 · 아카이브

이미 누군가 모아 한 덩어리로 배포해 둔 것을 내려받습니다. 과거 데이터를 한꺼번에 분석할 때 압도적으로 편합니다. 대신 실시간이 아닙니다. 갱신이 하루나 한 달 단위라 '오늘 아침 소식'에는 쓸 수 없습니다.

④ 서드파티 집계 서비스

여러 매체를 대신 모아 주는 상용 API를 돈 내고 씁니다. 소스를 직접 관리할 여력이 없고 예산이 있을 때 가장 빠릅니다. 대신 이 수업에서 배울 판단을 전부 남에게 맡기게 됩니다. 무엇을 기준으로 걸러졌는지 통제할 수 없고, 그 회사가 접으면 같이 멈춥니다.

⑤ 직접 스크래핑

사이트의 HTML을 내려받아 원하는 부분을 골라냅니다. 약속된 통로가 아니라 화면을 사람 대신 읽는 것입니다. 위의 네 칸이 전부 없을 때만 내려옵니다. 채용 공고나 관공서 게시판처럼 피드도 API도 없는 도메인이 실제로 있습니다. 대신 화면 구조가 바뀌면 조용히 깨지고, robots.txt와 약관을 확인해야 하며, 유지보수 비용이 다른 칸과 비교가 안 되게 큼니다.

우리는 RSS · Atom 피드를 골랐습니다. 우리가 다루는 것이 언론 기사이기 때문입니다. 1칸이 가장 좋지만 뉴스 매체는 대개 API를 열지 않습니다. 3칸은 실시간이 아니라 탈락, 4칸은 이 수업의 학습 목표 자체를 외주 주는 셈이라 탈락, 5칸은 유지보수 비용이 너무 큼니다. 남는 것이 2칸이고, 마침 언론사는 거의 예외 없이 RSS를 발행합니다.

다만 2칸만 쓰는 것은 아닙니다. Hacker News처럼 공개 API가 있는 곳은 1칸으로 가져옵니다. 칸은 소스마다 따로 고르는 것이지 프로젝트 전체에 하나만 고르는 것이 아닙니다.

조건이 달라지면 다른 선택지가 맞는 답이 됩니다. 무엇을 포기했는지 알고 고르면 됩니다.

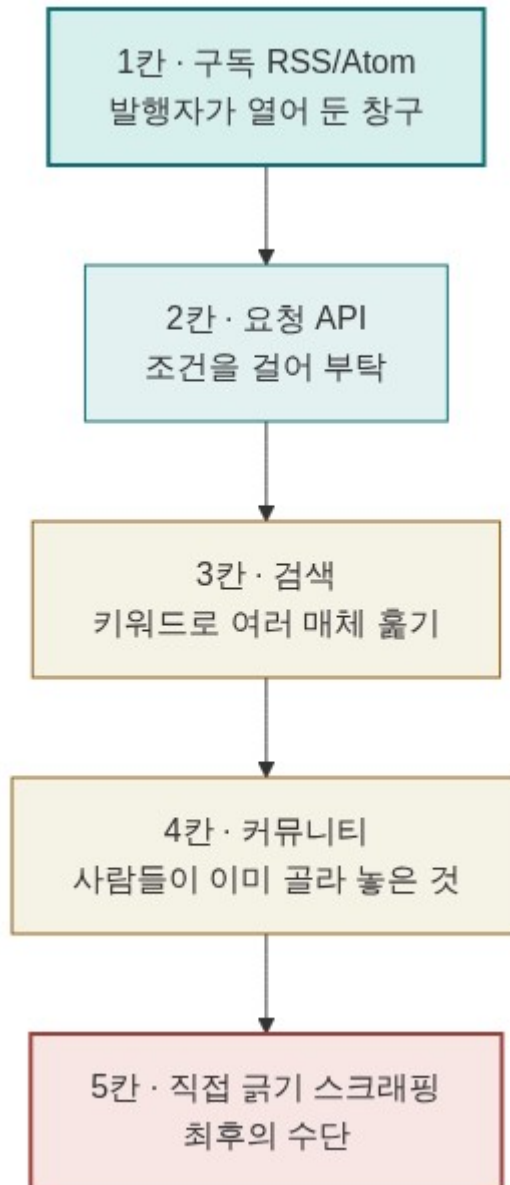


그림: 정보를 가져오는 다섯 가지 경로 — 위 칸이 가능하면 아래로 내려가지 않는다

객관식 퀴즈

Q. 사다리에서 '위 칸이 가능하면 아래로 내려가지 않는다'는 원칙의 근거로 가장 적절한 것은?

- 1. 위 칸일수록 데이터 양이 많아서 더 많은 기사를 모을 수 있기 때문이다
- 2. 위 칸일수록 발행자가 '가져가도 된다'고 밝힌 통로여서 형식 안정성과 법적 명확성이 함께 확보되기 때문이다
- 3. 위 칸일수록 코드가 짧아져서 개발 시간이 줄어들기 때문이다
- 4. 아래 칸은 기술적으로 어려워서 초보자가 구현할 수 없기 때문이다**제출**
- •

- 1. 위 칸일수록 데이터 양이 많아서 더 많은 기사를 모을 수 있기 때문이다
- •
- 2. 위 칸일수록 발행자가 '가져가도 된다'고 밝힌 통로여서 형식 안정성과 법적 명확성이 함께 확보되기 때문이다
- •
- 3. 위 칸일수록 코드가 짧아져서 개발 시간이 줄어들기 때문이다
- •
- 4. 아래 칸은 기술적으로 어려워서 초보자가 구현할 수 없기 때문이다

주관식 퀴즈

Q. 우리는 '언론 기사'라는 조건 때문에 2칸(RSS)을 골랐습니다. 그런데 만약 여러분이 만들려는 것이 경쟁사 제품의 가격 변동 알림이라면 사다리의 어느 칸으로 내려가게 될까요? 그리고 그때 포기하게 되는 것과 새로 짚어지게 되는 부담은 각각 무엇일까요?

0/300다음

더 해보기 — 내 분야는 몇 칸인지 조사하기

1. 여러분 분야의 사이트 세 곳을 골라 각각 몇 칸으로 가져올 수 있는지 조사한다

- 접근 힌트: 사이트 주소 뒤에 `/feed`, `/rss`, `/atom.xml` 을 붙여 보고, 개발자 문서나 API 안내 페이지가 있는지 찾아본다. 아무것도 없으면 5칸 후보다
- 성공하면: 여러분의 도메인이 이 수업의 구조를 그대로 쓸 수 있는 분야인지, 아니면 스크래핑이 불가피한 분야인지 판단할 수 있다

2. Hacker News 공개 API(1칸)를 호출해 RSS(2칸)와 무엇이 다른지 비교한다

- 접근 힌트: `https://hn.algolia.com/api/v1/search_by_date?tags=story&query=AI` 를 `requests` 로 부르면 JSON 이 온다. 점수(`points`)와 댓글 수(`num_comments`)가 함께 온다는 점을 본다
- 성공하면: 1칸은 RSS에 없는 비LLM 신호(추천 수 같은 것)를 준다는 것을 확인할 수 있다. 이 신호는 나중에 선별의 근거로 쓸 수 있어서 칸을 올리는 이득이 분명해진다

수집 경로를 고르는 데 참고할 자료

- [RSS 2.0 명세 \(Harvard\)](#).
- [Hacker News 검색 API \(Algolia\)](#). — 1칸(공식 API)의 실제 모양을 볼 수 있는 예

Chapter04

준비: 라이브러리 설치

실행

1

소스마다 무엇이 다른지 재 보기

섹션 3에서 2칸을 골랐습니다. 그런데 칸을 골랐다고 끝이 아니에요. 같은 2칸 안에서도 소스마다 품질이 천차만별이거든요.

말로 설명하는 대신 직접 재 봅시다. 쓸 만한 피드 여섯 곳에 실제로 요청을 보내 무엇을 돌려주는지 확인합니다. 볼 것은 세 가지예요 — 한 번에 몇 건을 주는지, 요약물 얼마나 주는지, 최신 글이 얼마나 오래됐는지.

실행 되었습니다.

실습: 소스 여섯 곳이 실제로 무엇을 주는지 재 보기 (키 불필요)

실행

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16
17
18
19
20
21
22
23
24
25

무엇을 보게 되나

여섯 줄짜리 표가 나오는데, 세 열이 저마다 제각각일 겁니다. 열별로 읽어 보세요.

건수 열 — 수백에서 천 단위를 한꺼번에 주는 곳이 있는가 하면, 딱 열 건만 주는 곳도 있습니다. 아카이브를 통째로 내주는 발행자와 최신 몇 건만 내주는 발행자가 섞여 있기 때문입니다. 열 건짜리 피드는 그 매체가 하루에 글을 열다섯 개 쓰면 우리가 읽기 전에 넘쳐서 사라 집니다. 하루 한 번만 수집하는 설계에서는 그날치를 놓친다는 뜻입니다.

요약길이 열 — 한 문장짜리부터 한 문단짜리까지 벌어지는데, 어느 쪽도 요약물 쓰기에는 부족합니다. RSS는 본문을 주는 통로가 아니라 "새 글이 났다"고 알리는 통로이기 때문입니다. 이것이 잠시 뒤 관문 이야기의 출발점이 됩니다.

최신글 열 — 며칠째 새 글이 없는 소스가 보일 겁니다. 24시간 창으로 수집하면 그런 곳은 대부분의 날에 0건을 기여합니다. 값이 음수로 나오는 소스도 있는데, 날짜만 기록하고 시각은 자정으로 채우는 발행자라 그렇습니다. 버그가 아니라 관행이에요.

1차 소스와 2차 소스

1차 소스는 사건의 당사자가 직접 발표하는 곳입니다. OpenAI·DeepMind·Hugging Face의 공식 블로그가 여기죠. 2차 소스는 그 발표를 받아 쓰거나 취재해 전하는 곳으로, TechCrunch·The Verge·AI타임스 같은 매체입니다.

둘을 같은 자리에 놓고 경쟁시키면 판단이 왜곡됩니다. OpenAI가 새 모델을 발표하면 매체 대여섯 곳이 같은 사건을 기사로 쓰거든요. 이때 '중요한 순서대로 다섯 개'를 고르면 같은 사건이 다섯 자리를 전부 차지할 수 있습니다.

그래서 1차 소스는 경쟁에서 빠고 따로 취급하는 방법이 있습니다. 소스 목록에 **tier** 값을 매겨 두고 **tier: 1** (당사자 발표)은 선별 경쟁 없이 통과시키는 식이죠. 당사자 발표는 '다른 기사보다 중요한가'를 물을 대상이 아니라 일어난 사실 그 자체니까요.

대신 무조건 통과시키면 그 회사의 홍보물까지 그대로 실립니다. 채용 공고나 행사 안내도 공식 블로그에 올라오거든요. 면제는 '경쟁 면제'이지 '검사 면제'가 아니어야 하고, 1차 소스가 많으면 면제만으로 자리가 다 차니 면제에도 상한이 필요합니다.

객관식 퀴즈

Q. 측정 결과에서 어떤 소스는 한 번 요청에 1,000건 넘게 주고 어떤 소스는 10건만 줍니다. 10건만 주는 소스를 다룰 때 가장 먼저 확인해야 할 것은?

- 1. 그 소스가 우리를 차단하고 있는지 User-Agent 를 바꿔 본다
- 2. 그 매체가 하루에 글을 몇 개 쓰는지 — 10건이 하루치보다 적으면 한 번 수집으로는 그 날치를 놓친다
- 3. 피드 주소에 `?limit=100` 을 붙여 더 많이 요청한다
- 4. 건수가 적으므로 관문 검사를 건너뛰어도 된다**제출**
- •
- 1. 그 소스가 우리를 차단하고 있는지 User-Agent 를 바꿔 본다
- •
- 2. 그 매체가 하루에 글을 몇 개 쓰는지 — 10건이 하루치보다 적으면 한 번 수집으로는 그날치를 놓친다
- •
- 3. 피드 주소에 `?limit=100` 을 붙여 더 많이 요청한다
- •
- 4. 건수가 적으므로 관문 검사를 건너뛰어도 된다

채택 기준 정하기

"괜찮아 보이는 곳"으로 정하면 몇 달 뒤 아무도 그 결정을 설명할 수 없게 됩니다. 소스를 평가하는 항목은 두 층으로 나뉩니다 — 통과하지 못하면 나머지를 볼 필요가 없는 것(관문)과 통과한 것들 사이의 우열(등급). 지금은 관문만 다룹니다.

관문 세 개

G1 본문 — 요약에 쓸 재료가 나오는가. 재료가 없으면 언어 모델이 제목만 보고 요약을 지어 냅니다. 환각의 가장 흔한 원인입니다.

G2 생존 — 최근에도 글이 올라오는가. 죽은 소스는 매일 0건을 기여하면서 요청 시간만 씁니다.

G3 접근 — 자동 수집이 막히지 않는가. 403이 돌아오거나 robots.txt가 금지하는 곳은 기술적으로 가능하더라도 채택하지 않습니다.

G1을 실제로 재 보겠습니다. 소스마다 최근 기사 세 건의 원문을 직접 가져와 확인합니다. 실행 되었습니다.

준비: 본문 추출 라이브러리

실행

1

실행 되었습니다.

실습: G1 본문 관문을 실제로 재 보기 (키 불필요, 1~2분 걸립니다)

실행

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17
18
19
20
21
22
23
24
25
26

무엇을 보게 되나

두 열을 나란히 놓고 보는 것이 핵심입니다.

RSS요약 열은 대체로 수백 자에 그칩니다 — 우리가 기준선으로 잡은 600자에 한 곳도 닿지 못하는 것이 보통입니다. 즉 RSS가 주는 것만으로 요약물 쓰게 하면 모델은 두세 문장을 읽고 한 문단을 지어내야 합니다. 지어내라고 시키면 지어냅니다.

원문추출 열은 대부분 전부 통과로 나옵니다. 기사 주소로 한 번 더 요청해 `trafilatura` 로 본문을 뽑으면 기준선을 넉넉히 넘깁니다. (유료 구독 기사나 동영상 중심 기사가 섞이면 거기서 실패가 나오는데, 그것이 바로 이 검사가 잡아야 할 겁니다.)

결론은 "이 소스들은 다 좋다"가 아니라 "RSS 수집과 본문 추출은 별개의 단계이고, 둘 다 있어야 G1을 통과한다" 입니다.

앞의 측정에서 요약길이가 소스마다 달랐던 것이 여기서 의미를 갖습니다 — 어차피 전부 부족하므로 요약길이는 소스를 고르는 기준이 못 됩니다. 고르는 기준은 '원문 주소로 갔을 때 본문이 뽑히느냐'입니다.

그리고 이 측정에는 비용이 있습니다. 소스마다 기사 세 건씩 추가 요청을 보내므로 수십 번의 왕복과 1~2분이 듭니다. 그래서 이 검사는 매일 돌리는 것이 아니라 소스를 채택할 때 한번, 그리고 가끔 재검할 때만 돌립니다.

이 결정을 검증하는 방법

확인할 것 — RSS 수집 + 원문 추출의 조합이 요약물 쓰기에 충분한 자료를 준다.

아래 신호로 확인합니다.

- **본문 추출 성공률** — 발행 후보로 올라온 기사 중 본문이 기준 길이를 넘긴 비율을 매 실행마다 기록합니다. **1.0 아래로 떨어지면** 어떤 소스가 최근에 구조를 바꿨거나 차단을 시작한 것입니다.
- **소스별 기여 건수** — 발행된 다섯 건이 각각 어느 소스에서 왔는지 매일 남깁니다. 한 달 내내 0건인 소스가 있으면 G2(생존)를 이미 잃은 것이므로 목록에서 뺍니다.
- **추출에 걸린 시간** — 본문 추출 단계의 소요 시간을 재 둡니다. 갑자기 늘어나면 어떤 소스가 응답을 지연시키고 있다는 뜻이고, 타임아웃이 없으면 전체가 매달립니다.

세 숫자 모두 분석 도구 없이 실행마다 한 줄씩 파일에 append 하면 충분합니다. 섹션 12에서 `store/metrics.jsonl` 에 이 값들을 직접 남겨 봅니다.

틀렸을 때의 증상 — 가장 먼저 나타나는 증상은 요약이 조금씩 부정확해지는 것입니다. 추출에 실패한 기사를 모델이 제목만 보고 요약하기 때문인데, 문장은 여전히 매끄러워서 읽는 사람은 알아차리지 못합니다. 그래서 사람 눈이 아니라 추출 성공률이라는 숫자를 봐야 합니다.

객관식 퀴즈

Q. G1(본문) 관문을 다른 항목들보다 먼저 두는 이유로 가장 적절한 것은?

- 1. 본문 길이가 가장 측정하기 쉬운 항목이기 때문이다
 - 2. 본문이 없으면 언어 모델이 제목만 보고 내용을 지어내게 되어, 이후 단계의 품질 문제가 전부 여기서 시작되기 때문이다
 - 3. 본문이 길수록 요약의 품질이 비례해서 좋아지기 때문이다
 - 4. trafilatura 라이브러리가 본문 추출을 가장 먼저 수행하도록 설계되어 있기 때문이다
- 제출**

• •

1. 본문 길이가 가장 측정하기 쉬운 항목이기 때문이다

• •

2. 본문이 없으면 언어 모델이 제목만 보고 내용을 지어내게 되어, 이후 단계의 품질 문제가 전부 여기서 시작되기 때문이다

• •

3. 본문이 길수록 요약의 품질이 비례해서 좋아지기 때문이다

• •

4. trafilatura 라이브러리가 본문 추출을 가장 먼저 수행하도록 설계되어 있기 때문이다



더 해보기 — 관문을 세 개 다 구현해 보기

1. G2(생존)를 실제로 재 본다 — 최근 14일 동안 글이 몇 건 올라왔는지 소스별로 센다

- 접근 힌트: `published_parsed` 로 14일 안쪽 항목만 세면 된다. 날짜가 아예 없는 항목이 섞여 있으니 그것부터 세어 보면 좋다
- 성공하면: DeepMind 처럼 '살아 있지만 뜸한' 소스와 '죽은' 소스를 구분하는 임계값을 직접 정해 보게 된다. 정답이 없다는 것이 요점이다

2. G3(접근)를 robots.txt 확인으로 구현한다

- 접근 힌트: 파이썬 표준 라이브러리 `urllib.robotparser` 의 `RobotFileParser` 를 쓴다. `can_fetch('*', url)` 한 줄이면 판정이 나온다
- 성공하면: 규칙을 읽는 코드가 열 줄이면 된다는 것과, 그럼에도 대부분의 프로젝트가 이것을 건너뛴다는 사실을 함께 확인하게 된다



소스를 검증할 때 참고할 자료

- [trafilatura 공식 문서](#) — 본문 추출에 쓴 라이브러리
- [Python 표준 라이브러리](#) — `urllib.robotparser`

Chapter05

수집 노드 만들기

소스가 정해졌으니 섹션 2의 빈 `collect` 를 채울 차례입니다. 할 일은 세 가지예요.

소스 목록을 돌며 글을 가져오고, 정해진 시간 창 안의 것만 남기고, 같은 글이 두 번 들어오지 않게 거릅니다. 소스 한 곳이 응답하지 않아도 나머지는 계속 모을 수 있게 예외처리까지 해보겠습니다.

바꾸는 건 `collect` 함수 하나뿐이고 State도 그래프도 건드리지 않습니다.

실행 되었습니다.

실습: 수집 노드를 채우고 그래프를 다시 돌리기 (키 불필요)

실행

- 1
- 2
- 3
- 4
- 5

6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

35
36
37
38
39
40
41
42
43
44

무엇을 보게 되나

첫 줄의 숫자가 0에서 벗어났고 나머지 네 줄은 여전히 0입니다. 고친 건 함수 하나뿐이고 `build()` 가 이름으로 노드를 찾으니까 새 `collect` 가 그대로 들어갑니다.

코드에서 세 군데를 보세요.

`if not at or at < cutoff` — 시간 창입니다. `hours` 는 State에서 오니까 나중에 이 값을 48로 바꾸면 같은 함수가 더 넓은 시간 동안 올라온 소스를 탐색합니다. 날짜가 아예 없는 항목도 여기서 함께 버려집니다.

`key = e.link.split("?")[0]` — 중복 제거입니다. 같은 글이 `?utm_source=...` 가 붙은 채로 두 번 들어오는 일이 흔해서 꼬리표를 떼고 비교합니다. 다만 이건 주소가 같은 것만 걸러 냅니다. 매체가 다른데 같은 사건을 다룬 기사는 그대로 통과합니다.

`except Exception: dead.append(name)` — 소스 하나의 실패를 격리합니다. 한 곳이 죽어도 나머지는 모이고, 죽은 곳은 로그에 남습니다. 이 마지막 한 줄이 없으면 어떻게 되는지가 이 섹션의 마지막 이야기입니다.

조용한 실패

실행이 멈추지도 않고 에러도 안 남기는데 결과만 틀리게 나오는 고장을 말합니다. 방금 코드에서 `dead` 를 로그에 남기지 않았다고 해 봅시다. 소스 한 곳이 조용히 빠지고, 파이프라인은 매일 성공적으로 끝나고, 건수가 조금 줄었다는 사실만 아무도 모릅니다.

자동으로 도는 시스템에서 가장 비싼 고장입니다. 요란하게 죽는 건 그날 바로 고치지만, 조용한 고장은 몇 달 뒤 누군가 "요즘 왜 그 매체 기사가 안 보이지?"라고 물을 때까지 남아 있고, 그때는 몇 달치를 거슬러 올라가야 하거든요.

태어나는 자리는 대개 정해져 있습니다. `continue` 와 `except: pass` 입니다. 둘 다 "이건 건너뛰다"는 뜻인데, 건너뛰는 사실을 아무 데도 적지 않으면 그 순간부터 증거가 사라져요. 위 코드가 `dead` 를 모아 로그에 넣는 이유입니다.

다만 모든 `continue` 에 기록을 붙이면 로그를 읽을 수 없게 됩니다. 기준은 그 건너뛰는 결과물을 바꾸는거예요. 소스 하나가 통째로 빠지는 건 바꾸고, 날짜 없는 항목 한 건이 빠지는 건 바꾸지 않습니다. 섹션 12에서 이 기록을 어디에 쌓을지 정합니다.

더 해보기 — 수집 노드를 흔들어 보기

1. SOURCES 에 소스를 한 곳 더 추가하고 총 건수가 얼마나 늘어나는지 본다

- 접근 힌트: Hugging Face 블로그는 <https://huggingface.co/blog/feed.xml> 이다. 추가 전후 건수를 적어 둔다
- 성공하면: 소스 하나가 하루에 기여하는 건수의 편차가 크다는 것을 알 수 있다. 이 숫자가 섹션 14에서 소스를 평가하는 근거가 된다

2. `INIT` 의 `hours` 를 6과 72로 바꿔 각각 몇 건이 모이는지 기록한다

- 접근 힌트: 세 번의 결과를 표로 적어 두고 요일을 바꿔 다시 한번 재 본다
- 성공하면: 주말과 평일의 차이가 시간 창보다 클 수 있다는 것을 확인할 수 있다. '기사가 모자라는 날'이 실재한다는 것을 알게 된다. 그런 날 우리 파이프라인은 적게 발행한다

3. 소스 주소 하나를 일부러 틀리게 바꿔 `dead` 가 찍히는지 확인한다

- 접근 힌트: 도메인에서 한 글자를 빼고 돌린다. 그다음 `dead.append(name)` 줄을 지우고 다시 돌려 본다
- 성공하면: 기록이 있을 때와 없을 때의 차이를 눈으로 본다. 없을 때는 건수가 줄었다는 것 말고 아무 단서가 없다

수집 노드를 만들 때 참고할 자료

- [feedparser — Common RSS elements](#) — `entries` 안에 어떤 항목이 들어오는지 정리돼 있다. 소스를 새로 추가하기 전에 그 피드가 무엇을 주는지 여기서 대조해 보면 좋다
- [feedparser — Date Parsing](#) — `published_parsed` 가 언제 비는지 적혀 있다. 위 코드가 `not at` 으로 거르는 항목이 바로 그런 경우다
- [MDN — URLSearchParams](#) — `?utm_source=...` 같은 꼬리표를 제대로 떼려면 문자열 자르기보다 이쪽이 안전하다. 중복 판정을 더 정확하게 만들고 싶을 때 본다

Chapter06

중요한 소식 선정하기

여러 가지 소식을 수집했을 때, 어떤 기준으로 소식을 선별하면 좋을까요?

가장 먼저 떠오르는 방법부터 해 봅시다. 기사마다 점수를 매기고 상위 다섯 개를 뽑는 겁니다.

여기서부터 OpenAI API 키가 필요합니다.

실행 되었습니다.

준비: 라이브러리 설치

실행

1

2

API 키 설정

키 설정 방법: 아래 세 가지 중 편한 것을 쓰세요.

- 로컬 주피터: 프로젝트 폴더에 .env 파일을 만들고 OPENAI_API_KEY=sk-... 를 적은 뒤 `from dotenv import load_dotenv; load_dotenv()` 를 실행합니다.
- Colab: 왼쪽 열쇠 아이콘(보안 비밀)에 OPENAI_API_KEY 를 등록하고 `from google.colab import userdata` 로 읽어 옵니다.
- 임시: `os.environ["OPENAI_API_KEY"] = "sk-..."` 로 직접 넣습니다. 다만 이 방식은 노트북에 키가 그대로 남으므로, 저장·공유 전에 반드시 지우세요.

실행 되었습니다.

준비: API 키 설정 (LMS에서는 여기에 직접 입력)

실행

1

2

3

4

5

6

7

- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15
- 16
- 17
- 18

실행 되었습니다.

실습: 기사마다 독립적으로 점수 매기기

실행

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15

16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

무엇을 보게 되나

마지막 두 줄, 점수 분포와 서로 다른 점수 값의 개수를 보세요.

열두 건을 채점했는데 서로 다른 값은 네댓 개밖에 안 나올 겁니다. 그리고 그 값들이 고르게 퍼지지 않고 가운데 한두 점에 뭉칩니다. 1~10점을 주라고 했는데 실제로는 1점이나 10점이 거의 안 나오고 중간대만 쓰는 것입니다.

그래서 무슨 일이 생기는지 중요합니다. 다섯 건을 뽑아야 하는데 경계가 동점 위에 떨어 집니다.

같은 점수를 받은 서너 건 중 몇 개만 골라야 하는 상황이 되고, 점수는 그 근거를 주지 않습니다. 결국 목록 순서처럼 기사의 가치와 무관한 것으로 자르게 됩니다.

그리고 이것은 열두 건일 때의 이야기입니다. 실제로는 하루에 100건 넘게 들어오는데 점수 값은 여전히 열 개뿐이라 한 점수에 수십 건씩 몰립니다. "7점 이상"으로 자르면 몇 건이 나올지 실행 전에는 알 수 없고, 어떤 날은 세 건, 어떤 날은 마흔 건이 됩니다.

진짜 문제는 정확도가 아닙니다 — 우리에게 필요한 것은 "이 기사가 몇 점인가"가 아니라 "오늘 것 중 위에서 다섯 번째까지가 무엇인가"인데, 독립 채점은 후자를 묻지 않습니다.

선택지

정해야 할 것 — 100여 건에서 다섯 건을 고르려면 언어 모델에게 무엇을 물어야 하는가?

네 가지 구조가 있고, 각각 비용과 판정의 질이 다릅니다.

① 독립 채점 후 상위 N

기사 하나씩 따로 점수를 매기고 정렬해 자릅니다. 기준이 절대적이라 날짜가 달라도 비교할 수 있고, 각 판정이 독립적이라 병렬 처리가 자유롭습니다. 대신 방금 본 동점 몰림이 생기고, 호출 횟수가 기사 수에 비례해 가장 비쌉니다.

② 토너먼트 (쌍대 비교)

두 건씩 짝지어 "둘 중 어느 쪽이 더 중요한가"를 묻고 이긴 쪽을 올립니다. 사람이 가장 잘하는 형태의 판단이라 판정의 질이 높고, 동점이 원리적으로 없습니다. 대신 호출 횟수가 폭발합니다. 100건이면 최소 99번, 제대로 순위를 매기려면 수백 번입니다.

③ 한 번에 전부 비교

후보 전체를 한 프롬프트에 넣고 "이 중 상위 다섯 개"를 한 번에 묻습니다. 호출이 한 번뿐이라 가장 싸고 빠르며, 모든 후보를 동시에 보므로 상대적 판단이 가능합니다. 대신 100건을 넣으면 컨텍스트가 길어져 중간에 있는 것을 놓치고, 출력이 길어질수록 형식이 깨질 확률도 올라갑니다.

④ 2단 상대평가 (예선 → 본선) ← 우리가 고른 것

후보를 묶음으로 나눠 각 묶음에서 소수를 뽑고(예선), 살아남은 것만 다시 한 번에 비교합니다(본선). 묶음을 모델이 감당할 크기로 유지하면서도 마지막 판단은 한 화면에서 이루어집니다. 호출 수도 기사 수에 비례하지 않습니다. 대신 예선에서 떨어진 것은 본선을 못 봅니다. 강한 기사들이 한 묶음에 몰리면 일부는 다른 묶음의 약한 기사보다 먼저 탈락합니다.

우리는 2단 상대평가 (예선 → 본선)를 골랐습니다. ①은 동점 때문에 '다섯 건'이라는 요구를 만족시키지 못하고, ②는 비용이 감당이 안 되며, ③은 후보가 늘어나면 무너집니다. ④는 ③의 장점(상대적 판단)을 유지하면서 컨텍스트 길이 문제만 분할로 풀 것입니다. ④의 약점인 '묶음 윤'은 예선을 넉넉히 통과시켜(묶음당 8건) 완화합니다.

조건이 달라지면 다른 선택지가 맞는 답이 됩니다. 무엇을 포기했는지 알고 고르면 됩니다.

③을 실제로 돌려 보기

방금 채점한 것과 똑같은 12건을 한 프롬프트에 전부 넣고 "이 중 상위 다섯 개"를 한 번에 물어보겠습니다. 독립 채점과 결과가 얼마나 다를지 예상해 보고 실행하세요.

실행 되었습니다.

실습: 같은 12건을 한 화면에 놓고 비교하기

실행

1

2

3

4

5

6

7

8

9

10

11

12

13

14

무엇을 보게 되나

두 가지를 확인해 보세요.

첫째, 고른 다섯 건이 독립 채점과 크게 다르지 않을 것입니다. 명단이 거의 겹칩니다. 그러면 상대평가는 쓸모없는 것일까요. 그렇지 않습니다 — 순서가 바뀝니다. 독립 채점에서 가장 높은 점수를 받았던 기사가 상대평가에서는 중간으로 내려가고, 동점에 묻혀 있던 기사가 1 위로 올라오는 일이 흔합니다. '절대 점수'와 '이 중 무엇이 제일 중요한가'는 다른 질문이고, 우리에게 필요한 것은 후자입니다.

둘째가 훨씬 중요합니다. 다섯 건 중에 같은 사건을 다룬 기사가 둘 이상 들어 있지 않은지 보는 것입니다.

큰 발표가 있었던 날이면 매체 여럿이 같은 일을 기사로 쓰고, 그것들이 나란히 뿔뿔히 올라옵니다. 같은 인물의 같은 발언, 같은 회사의 같은 출시가 두 자리를 차지하는 식입니다.

후보 구성에 따라 이번 실행에서는 안 보일 수도 있습니다. 그러면 다른 날 다시 돌려 보세요.

'같은 기사를 어떻게 알아볼 것인가'는 수집에서 반만 풀었습니다. 나머지를 선별 단계로 미뤘습니다. 수집에서는 주소가 같은 것만 걸러 내고, '같은 사건'은 "한 화면에 놓고 보면 모델이 알아서 걸러 주겠지"라고 기대한 것입니다. 그 기대가 여기서 무너집니다. 모델은 후보 전체를 보고 있었는데도 둘 다 골랐습니다.

가장 간단하게는 "같은 사건을 다룬 기사는 하나만 고르세요."와 같이 프롬프트를 보강하는 방법이 있습니다. 더 강건하게 중복제거를 위해서는 코드로 강제하는 방법이 있습니다.

상대평가와 절대평가

절대평가는 대상 하나를 보고 미리 정한 척도에 비추어 값을 매기는 것이고, 상대평가는 여러 대상을 함께 놓고 서로 견주어 순서를 정하는 것입니다. 같은 모델, 같은 기준이라도 무엇을 묻느냐에 따라 다른 답이 나와요.

우리가 답해야 하는 질문이 상대적이라서 상대평가를 씁니다. 요구사항은 "7점 이상을 전부 실어라"가 아니라 "오늘 것 중 다섯 개를 골라라"거든요. 출력 개수가 고정되어 있으면 그건 정렬 문제이지 채점 문제가 아닙니다. 채점으로 정렬 문제를 풀면 동점 물림을 만나죠.

방법은 후보들을 한 프롬프트에 나란히 넣고 순위를 매기게 하는 겁니다. 모델이 각 후보를 다른 후보와의 관계 속에서 판단하니 "오늘은 큰 뉴스가 없으니 이 정도도 3위" 같은 판단이 가능해져요. 절대 점수로써는 나올 수 없는 판단입니다.

대신 결과가 그날의 후보 구성에 의존합니다. 같은 기사가 어제는 1위, 오늘은 6위가 될 수 있고 그게 정상이에요. 그래서 날짜를 건너뛴 비교가 불가능하고, 품질 추이를 보려면 점수가 아니라 깔때기 숫자와 소스 분포 같은 지표를 남겨야 합니다(섹션 13). 후보가 많아지면 한 화면에 다 넣을 수 없다는 한계도 그대로 남는데, 그걸 푸는 게 다음 섹션입니다.

객관식 퀴즈

Q. 독립 채점에서 12건에 점수 값이 5개뿐이었던 것이 문제가 되는 이유로 가장 적절한 것은?

- 1. 모델의 판단이 부정확해서 비슷한 기사들을 구분하지 못했기 때문이다
- 2. 우리가 풀어야 하는 문제가 '개수가 고정된 정렬'인데 채점은 그 질문에 답하지 않아, 경계에서 자를 근거가 사라지기 때문이다
- 3. 1~10점 척도가 너무 좁아서 1~100점으로 바꾸면 해결되기 때문이다
- 4. temperature를 0으로 두어 모델이 다양한 값을 내놓지 못했기 때문이다

• •

1. 모델의 판단이 부정확해서 비슷한 기사들을 구분하지 못했기 때문이다

• •

2. 우리가 풀어야 하는 문제가 '개수가 고정된 정렬'인데 채점은 그 질문에 답하지 않아, 경계에서 자를 근거가 사라지기 때문이다

• •

3. 1~10점 척도가 너무 좁아서 1~100점으로 바꾸면 해결되기 때문이다

• •

4. temperature를 0으로 두어 모델이 다양한 값을 내놓지 못했기 때문이다

주관식 퀴즈

Q. 상대평가는 후보 열두 건을 모두 한 화면에서 보고 있었는데도 같은 사람의 같은 발언을 다룬 기사 두 건을 3위와 4위에 나란히 올렸습니다. 모델이 '실수'한 것일까요? 모델 입장에서 이 선택이 왜 자연스러운 것인지 설명하고, 우리가 프롬프트에 무엇을 적었어야 했는지 함께 답해 보세요.

0/300다음

더 해보기 — 네 가지 구조를 직접 비교하기

1. 같은 12건에 토너먼트(②)를 구현해 호출 횟수를 세어 본다

- 접근 힌트: `itertools.combinations` 로 모든 쌍을 만들고 각 쌍마다 한 번씩 물으면 66번이다. 승수를 세어 순위를 매긴 뒤 ①·③의 결과와 비교한다
- 성공하면: 판정의 질과 비용이 정확히 어떤 비율로 교환되는지 숫자로 확인할 수 있다. 66번의 호출이 다섯 건을 고르는 데 값한 일인지 스스로 판단하게 된다

2. 독립 채점을 temperature=0.7 로 세 번 반복해 같은 기사의 점수가 흔들리는지 본다

- 접근 힌트: 같은 제목에 대해 세 번 호출하고 표준편차를 계산한다
- 성공하면: 절대 점수라는 것이 얼마나 불안정한 값인지 체감할 수 있다. 이 실험 결과가 '점수를 기록해 두고 나중에 비교하자'는 계획의 타당성을 판단하는 근거가 된다

Chapter07

후보가 수백 건이 되면

섹션 6의 상대평가는 후보가 열두 건이라 한 프롬프트에 무리 없이 들어갔습니다. 그런데 실제 소스 열 곳 남짓을 다 켜면 하루에 수백 건이 들어옵니다.

한 프롬프트에 넣으면 토큰 한도에는 들어갈 수도 있어요. 문제는 다른 데 있습니다. 목록이 길어질수록 모델은 앞뒤를 잘 보고 중간을 흘립니다.

그래서 나누되 마지막 판단만은 한 화면에서 하도록 만듭니다. 이게 예선과 본선이에요. 먼저 완성된 파이프라인이 실제로 어떻게 좁혀 가는지 봅시다.

단계별로 얼마나 좁혀지나

완성된 파이프라인을 돌리면 단계마다 남은 건수가 한 줄씩 찍힙니다. 모양이 이렇습니다.

- ① 수집 전체 → 창(24h) → 중복 제거 → 소스별 상한 → 남은 후보
 - 예선 묶음 ...건 → 8건
 - 예선 묶음 ...건 → 8건
- ② 예선 통과 (tier1 자동통과 포함)
- ③ 본선 → 최대 5건
 - [탈락한 기사] 왜 떨어졌는지 한 줄
- ⑥ 검수 → 합격·불합격

단계마다 무엇이 얼마나 빠지는지를 따라가 보세요. 값은 그날그날 다르지만 비율의 모양은 대체로 비슷합니다.

수집 → 시간 창. 24시간으로 자르면 크게 줄어듭니다. 아카이브를 통째로 주는 소스가 섞여 있으면 여기서 대부분이 떨어집니다.

시간 창 → 중복·상한. 여기서 중복으로 빠지는 건수가 놀랄 만큼 적다는 것에 주목하세요. 주소를 맞춰 보는 방식이 잡을 수 있는 것이 딱 그 정도이고, '같은 사건'은 여기서 하나도 걸러지지 않습니다. 소스별 상한으로 빠지는 쪽이 훨씬 많습니다.

→ 예선. 후보가 묶음으로 쪼개져 각 묶음에서 여덟 건씩 올라오고, 1차 소스 자동 통과분이 더해집니다. 묶음 크기가 설계 결정입니다 — 모델이 한 화면에서 흘리지 않고 볼 수 있는 크기로 정한 값입니다.

→ 본선. 예선 통과분이면 한 화면에 놓을 수 있으므로 섹션 6의 상대평가를 그대로 씁니다. 탈락한 것들에 이유가 붙어 있는 것이 중요합니다 — 이유가 없으면 선별이 잘못됐을 때 무엇을 고칠지 알 수 없습니다.

전체 LLM 호출은 열 번 남짓입니다. 후보를 전부 독립 채점했다면 후보 수만큼 필요했을 것입니다.

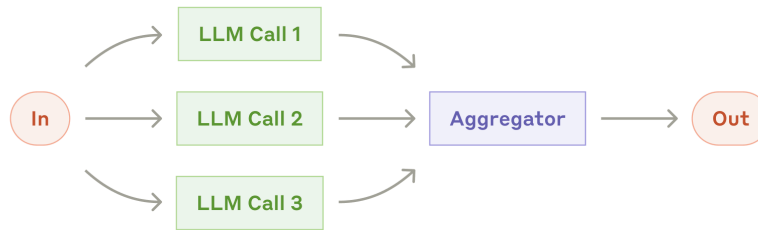


그림: 병렬화(sectioning) — 입력을 나눠 동시에 처리하고 결과를 하나로 합친다 (출처: Anthropic)

위 그림은 Anthropic이 정리한 병렬화(parallelization) 패턴, 그중에서도 입력을 쪼개 나눠 맡기는 sectioning 변형입니다. 우리 예선이 정확히 이 모양입니다. 후보를 묶음으로 쪼개 (In 에서 갈라지는 화살표) 각 묶음을 서로 모르는 채로 처리한 뒤(LLM Call 1, LLM Call 2) 결과를 하나로 모읍니다(Aggregator).

"서로 모르는 채로"가 중요합니다. 제약처럼 보이지만 장점이기도 합니다. 묶음을 몇 개로 나누든 코드가 같고 하나가 실패해도 나머지는 살아남기 때문입니다.

대신 대가가 있습니다. 묶음 사이의 비교는 예선에서 일어나지 않습니다. 40건 묶음의 9등이 22건 묶음의 3등보다 좋아도 전자가 탈락합니다. 이 손실을 줄이기 위해서 예선을 넉넉하게 통과시킬 수 있습니다. 최종 5건보다 훨씬 많은 수인 묶음 당 8건을 남깁니다.

예선 만들기

섹션 6과 다른 점이 하나 있습니다 — 출력을 정해진 형식으로 받는 것입니다.

섹션 6에서는 "번호만 심표로"라고 부탁했고 운 좋게 지켜졌지만 이것은 부탁이지 보장이 아닙니다. 모델이 "3, 7, 그리고 9번도 좋습니다"라고 답하면 `int(x)` 에서 오류가 발생해버립니다. 구조화 출력(structured outputs) 을 쓰면 형식이 스키마로 강제됩니다.

실행 되었습니다.

실습: 구조화 출력으로 예선 한 묶음 돌리기

실행

1

2

3

4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25

무엇을 보게 되나

이번에는 기사 제목 앞에 **[사건 라벨]** 이 달리고, 그 아래 고른 이유가 한 줄씩 붙습니다.

라벨 목록을 먼저 훑어보세요. 다섯 건의 라벨이 전부 다르다면, 섹션 6에서 나란히 뽑혔던 같은 사건 기사가 한 건으로 줄어든 것입니다. 같은 라벨이 두 번 나왔다면 아직 줄지 않은 것이고요. 어느 쪽이든 다음 섹션으로 이어집니다.

구조화 출력

모델의 응답을 자유 문장이 아니라 미리 정의한 스키마에 맞춰 받는 기능입니다.

위 코드에서는 `Pick` 과 `Shortlist` 라는 pydantic 모델을 정의하고 `response_format` 으로 넘겼고, 응답은 반드시 그 모양으로 옵니다.

자유 문장을 파싱하는 코드는 언젠가 반드시 깨지거든요. "번호만 쉼표로"라고 부탁해도 모델은 가끔 설명을 덧붙이고 그때 `int(x)` 가 터집니다. 매일 자동으로 도는 시스템에서 '가끔'은 '한 달에 몇 번'이고, 새벽에 조용히 실패합니다.

스키마가 출력 형식을 제약하니 파싱 자체가 사라집니다. 부수적인 이득이 더 커요.

스키마의 필드 이름과 `description` 이 지시의 일부로 작동하거든요. `event` 필드에 "같은 사건이면 같은 라벨"이라고 적은 게 그것이고, 덕분에 모델의 내부 판단이 우리가 셀 수 있는 값으로 밖에 나왔습니다.

형식은 보장되지만 내용은 보장되지 않습니다. 스키마는 `event` 필드가 채워지게 하지만 같은 사건에 같은 라벨이 붙는다는 건 보장하지 않아요. 그건 여전히 프롬프트에 적은 부탁이고 부탁은 확률적으로 지켜집니다. 다음 섹션에서 확인할 거예요. 그리고 필드를 늘릴수록 출력 토큰이 늘어 비용과 지연이 커지니 나중에 실제로 쓸 필드만 넣으세요.

선별 노드 작성하기

예선을 한 묶음 돌려 봤으니 이제 노드로 만듭니다. 묶음을 전부 돌고, 살아남은 것을 한 화면에 놓고 다시 고르는 것까지가 `select` 하나입니다.

실행 되었습니다.

실습: 선별 노드를 채우고 그래프를 다시 돌리기

실행

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11

12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30

무엇을 보게 되나

앞의 두 줄에 숫자가 찻습니다. 수집이 몇 건을 모았고, 예선이 몇 건으로 줄였고, 본선이 다섯 건을 남겼는지가 한 줄에 보입니다.

프롬프트에 "같은 사건은 하나만"이라고 적었는데도 비슷한 기사가 나란히 올라올 수 있습니다. 프롬프트는 요청이지 보장이 아니거든요. 정말 막고 싶으면 아래 더 해보기에서 코드로 세어 봅니다.

이 결정을 검증하는 방법

확인할 것 — 후보를 묶음으로 나눠 예선을 치르고 통과분만 본선에 올리는 구조가 좋은 기사를 놓치지 않는다.

아래 신호로 확인합니다.

- **묶음별 통과선의 높이** — 각 묶음에서 마지막으로 통과한 기사(8등)와 처음 탈락한 기사(9등)를 기록합니다. 묶음마다 8등의 수준이 크게 다르면 묶음 운이 작동하고 있다는 뜻입니다.
- **본선 탈락자의 탈락 이유** — 본선에서 떨어진 기사와 그 이유를 매번 남깁니다. "중복"이나 "애매함"이 반복되면 예선이 아니라 앞 단계를 고쳐야 합니다.
- **예선 통과 수 대비 최종 발행 수** — 예선 통과분 가운데 최종 발행된 비율을 봅니다(대체로 3분의 1 안팎입니다). 1에 가까워지면 예선이 뽕뽕해 본선이 고를 여지가 없다는 뜻이고, 너무 낮으면 예선이 헐거워 본선에 부담을 떠넘긴다는 뜻입니다.

세 가지 모두 파이프라인이 이미 출력하고 있는 값입니다. 위 기록의 **예선 묶음 40건 → 8건** 과 본선 탈락 사유 두 줄이 그것입니다. 화면에 찍는 것을 파일로도 남기면 됩니다(섹션 13).

틀렸을 때의 증상 — 예선이 잘못되고 있을 때의 증상은 뉴스레터가 심심해지는 것입니다. 큰 사건이 없는 날의 결과물과 구분이 안 되므로 "요즘 업계가 조용한가 보다"로 오해하게 됩니다. 그래서 결과물이 아니라 탈락자 목록을 봐야 합니다 — 거기에 좋은 기사가 쌓이고 있다면 문제는 업계가 아니라 우리 예선입니다.

객관식 퀴즈

Q. 예선을 40건 묶음으로 나눌 때 각 묶음이 '서로 모르는 채로' 처리되는 것에 대한 설명으로 가장 적절한 것은?

- 1. 묶음 사이의 정보 공유가 없으므로 반드시 순차적으로 실행해야 한다
- 2. 묶음 간 비교가 불가능해 좋은 기사를 놓칠 수 있지만, 대신 묶음 수와 무관하게 코드가 같고 하나가 실패해도 나머지가 살아남는다
- 3. 묶음을 나누면 전체를 한 번에 넣는 것보다 항상 더 좋은 결과가 나온다
- 4. 묶음마다 서로 다른 기준을 적용해야 다양성이 확보된다**제출**

• •

1. 묶음 사이의 정보 공유가 없으므로 반드시 순차적으로 실행해야 한다

• •

2. 묶음 간 비교가 불가능해 좋은 기사를 놓칠 수 있지만, 대신 묶음 수와 무관하게 코드가 같고 하나가 실패해도 나머지가 살아남는다

• •

3. 묶음을 나누면 전체를 한 번에 넣는 것보다 항상 더 좋은 결과가 나온다

• •

4. 묶음마다 서로 다른 기준을 적용해야 다양성이 확보된다

🚀 더 해보기 — 묶음 크기가 결과를 바꾸는지 확인하기

1. 같은 후보를 묶음 크기 10·20·40으로 나눠 예선을 돌리고 통과자 명단을 비교한다

- 접근 힌트: 후보를 섞는 순서를 고정(`random.seed`)해 두고 묶음 크기만 바꾼다. 세 결과의 교집합과 차집합을 출력한다
- 성공하면: '묶음 운'이 실제로 얼마나 결과를 바꾸는지 눈으로 볼 수 있다. 차집합이 크다면 예선 통과 수를 늘려야 한다는 근거가 된다

2. 구조화 출력을 쓰지 않은 섹션 6 방식으로 20번 반복해 형식이 깨지는 횟수를 센다

- 접근 힌트: `try/except ValueError` 로 감싸고 실패 횟수를 누적한다
- 성공하면: "가끔 깨진다"가 실제로 몇 퍼센트인지 숫자로 알게 된다. 그 숫자가 구조화 출력을 도입할지 판단하는 근거가 된다

3. 같은 사건이 두 번 실리는지 세어 보고, 거슬리면 코드로 막아 본다

- 접근 힌트: `Counter(p.event for p in finals)` 로 라벨이 겹치는지 센다. 막으려면 본선 결과를 순서대로 훑으며 이미 나온 라벨은 건너뛰고 다음 후보를 올리면 된다. 같은 방식으로 매체 상한도 걸 수 있다
- 성공하면: 프롬프트에 적은 부탁이 몇 번 중 몇 번 지켜지는지 숫자로 알게 된다. 그리고 지켜야 하는 규칙은 결국 코드로 세는 수밖에 없다는 것도

Chapter08

제목과 링크만 있는 상태

지금까지 수집한 내용에는 RSS가 주는 제목과 링크밖에 없습니다. 그래프를 돌려 보면 앞의 두 줄에만 숫자가 있고 ③ 취재 부터는 0이죠. 결과물도 제목 다섯 개뿐이라 받아 보는 사람 입장에서는 링크 목록입니다.

실제로 그 링크에 가서 본문을 뽑고, 읽을 만한 요약으로 바꾸는 과정을 노드로 만들어 봅시다.

섹션 4에서 G1 관문을 만들며 "재료가 없으면 언어 모델이 제목만 보고 요약을 지어낸다"고 했는데, 그 재료를 쓰는 곳이 여기입니다.

그런데 코드를 쓰기 전에 정할 게 하나 있어요. 요약을 어떤 모양으로 받을 것인가입니다.

요약을 어떤 칸으로 받을 것인가

"기사를 요약해 줘"라고 하면 문단 하나가 돌아옵니다. 그걸 그대로 쓰면 될 것 같지만, 한 덩어리로 받는 순간 다음 단계에서 할 수 있는 일이 크게 줄어듭니다.

우리가 받을 것은 세 가지입니다. 카드에 들어갈 **헤드라인**, 세 문장짜리 **요약**, 그리고 **"왜 중요한가"** 한 문장. 셋으로 나누는 이유는 성질이 다르기 때문이에요.

headline 과 **summary** 는 원문에 있는 내용을 옮긴 것이라 원문과 대조해 검사할 수 있습니다. 반면 **why** 는 "국내 개발팀에게 왜 중요한가"라는 독자 관점의 해석이라 원문 어디에도 없어요.

한 덩어리로 받으면 어떻게 될까요. 다음 섹션에서 "이 문단이 원문에 근거하는가"를 물을 텐데, **why** 에 해당하는 부분이 원문에 없으니 매번 불합격합니다. 임계값을 낮추면 이번엔 진짜 환각도 통과하고요. 한 덩어리로 받는 순간 그 구분 자체가 불가능해집니다.

그래서 결론을 먼저 적어 둡니다. 출력 스키마를 정하는 일은 형식을 정하는 일이 아니라, 나중에 무엇을 자동으로 검사할 수 있을지를 미리 정하는 일입니다.

선택지

정해야 할 것 — 생성 단계의 결과물을 어떤 구조로 받을 것인가?

네 가지가 있고, 아래로 갈수록 검증하기는 쉬워지지만 비싸지거나 자유도가 줄어듭니다.

① 자유 텍스트 한 덩어리

"뉴스레터 항목을 써 줘"라고 하고 문단 하나를 받습니다. 가장 자유롭고 프롬프트도 짧습니다. 결과물을 사람이 읽고 쓰는 단계라면 이것으로 충분합니다. 대신 어디까지가 사실이고 어디부터가 해석인지 코드가 알 수 없습니다. 검사도, 부분 재작성도, 다른 형식으로 재조립도 전부 막힙니다.

② 검증 방법이 다르면 칸을 나눈다 ← 우리가 고른 것

원문과 대조할 것(**headline** · **summary**), 대조 대상이 없는 것(**why**)을 각각 다른 칸으로 받습니다. 칸마다 다른 검사를 붙일 수 있습니다. 불합격 사유도 칸 단위로 나오므로 어디를 고쳐야 할지 분명합니다. 우리처럼 뒤에 검수 단계가 붙는 파이프라인에 맞습니다. 대신 칸이 늘수록 출력 토큰과 지연이 늘고, 모델의 주의가 분산되어 개별 품질이 떨어질 수 있습니다.

③ 근거 구절까지 함께 받는다

각 문장마다 원문의 근거 구절을 함께 내놓게 합니다. 검증이 결정론적이 됩니다. 근거를 못 대면 그 자리가 바로 환각입니다. 사실성이 최우선인 문서에 맞습니다. 대신 출력이 길어져 비용이 늘고, 요약이 원문 표현에 묶여 자유도가 크게 떨어집니다. 게다가 모델이 근거 구절 자체를 지어낼 수 있어 결국 문자열 확인이 또 필요합니다.

④ 단계를 쪼개 여러 번 호출한다

요약을 받고, 별도 호출로 '왜 중요한지'를 받고, 또 다른 호출로 분류를 받습니다. 호출마다 프롬프트가 단순해져 지시 준수율이 올라갑니다. 단계별로 모델을 달리 쓸 수도 있습니다. 대신 호출 수만큼 비용과 지연이 곱해지고, 앞 호출의 결과를 뒤 호출에 실어 보내야 해서 전체 토큰도 늘니다.

우리는 '검증 방법이 다르면 칸을 나눈다' 쪽을 골랐습니다. 우리 파이프라인에는 뒤에 검수 단계가 있기 때문입니다. 검수가 없다면 ①로 충분하고, 실제로 사람이 읽고 다듬어 보내는 뉴스레터는 ①로 만듭니다. ③은 매력적이지만 '세 문장으로 건조하게'라는 우리 요구와 충돌하고, ④는 기사 다섯 건에 호출이 열다섯 번이 되어 과합니다.

칸을 나눌 때 기준은 "이 칸은 무엇과 대조할 수 있는가" 입니다. 원문과 대조할 수 있는 것, 정해진 목록과 대조할 수 있는 것(토픽 분류), 대조 대상이 없는 것(해석) — 성질이 다르면 칸을 나눕니다. 섹션 7에서 `event` 라벨을 따로 뽑게 한 것도 같은 이유였습니다.

조건이 달라지면 다른 선택지가 맞는 답이 됩니다. 무엇을 포기했는지 알고 고르면 됩니다. 실행 되었습니다.

준비: 라이브러리 설치

실행

1

2

실행 되었습니다.

실습: 기사 세 건을 취재해 본다

실행

1

2

3

4

5

6

7

8

9

10

11

12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35

무엇을 보게 되나

기사 세 건의 헤드라인과 요약이 차례로 나옵니다.

내용만 보면 대체로 정확할 겁니다. 본문을 제대로 읽었고 사실을 왜곡하지도 않았어요. 그런데 언어를 보세요. 영어 기사를 요약한 결과가 한국어가 아니라 영어로 나온 것이 섞여 있

을 가능성이 높습니다.

한국어를 요구하지 않은 게 아니에요. 시스템 프롬프트에 "국내 개발팀을 위한 AI 뉴스레터 기자"라고 적었고, 스키마의 필드 설명에도 "한국어 헤드라인", "~합니다체"라고 명시했습니다. 두 군데나 적었는데 안 지켜집니다.

입력이 영어라서 그렇습니다. 모델은 본문의 언어를 따라가는 경향이 강하고, 그 경향이 우리 지시보다 세요. 본문이 짧은 기사일수록 지시의 비중이 커져 한국어가 나오기도 하는데, 그것도 우리가 통제할 결과는 아닙니다.

섹션 7에서 본 것과 같습니다. 프롬프트는 요청이지 보장이 아니에요. 그러면 무엇을 코드로 옮길지는 "지켜졌는지 출력만 보고 확인할 수 있는가"로 갑니다. 언어는 확인할 수 있죠. 한글이 한 글자라도 있는지 세면 되니까요. 그러니 코드로 강제할 수 있는 규칙입니다.

출력 스키마와 검증 가능성

생성 결과를 어떤 칸에 나눠 담을지 미리 선언하는 것이고, 그 선언이 곧 나중에 무엇을 검사할 수 있는지의 목록입니다. `pydantic` 모델로 필드를 정의하고 구조화 출력으로 받으면 모델이 그 형태를 채웁니다.

검사할 수 없는 결과물은 사람이 매번 읽어야 하는 결과물이에요. 자동으로 도는 시스템에서 그건 자동화가 아니죠. 구조가 없으면 부분 재작성도 불가능합니다. 어디가 문제인지 지목할 수 없으면 "다시 써"라고만 할 수 있고, 그러면 멀쩡했던 부분까지 바뀌거든요.

칸을 나누는 기준은 대조 대상입니다. 원문과 대조하는 칸, 정해진 목록과 대조하는 칸, 대조 대상이 없는 칸을 각각 분리해요. 대조 대상이 없는 칸은 정의를 바꿔 대조 가능하게 만들 수도 있습니다. `why` 에게 "새로운 사실을 추가하지 말고 요약에 있는 내용만 근거로 삼으라"고 지시하면 대조 대상이 원문이 아니라 `summary` 가 되니까요.

쓰지도 검사하지도 않을 칸은 비용만 냅니다. '있으면 좋을 것 같아서' 넣은 필드가 대표적이죠. 그리고 스키마가 보장하는 건 칸이 채워진다는 것이지 채워진 내용이 지시를 지켰다는 게 아닙니다. 섹션 8에서 `headline` 필드 설명에 '한국어'라고 적었는데도 영어가 들어온 게 그 증거예요. 형식은 스키마가 보장하고 내용은 검사가 확인해야 합니다.

객관식 퀴즈

Q. 시스템 프롬프트에 '국내 개발팀을 위한'이라고 쓰고 스키마에도 '한국어'라고 명시했는데도 세 건 중 두 건이 영어로 나왔습니다. 가장 적절한 처방은?

- 1. 프롬프트에 '반드시 한국어로'를 대문자와 느낌표로 강조해서 다시 적는다
- 2. 출력에 한글이 포함됐는지 코드로 검사하고, 아니면 재요청하거나 번역 단계를 붙인다
- 3. 더 큰 모델로 바꾸면 지시를 더 잘 따르므로 모델을 교체한다
- 4. 영어 기사는 애초에 수집하지 않도록 소스 목록에서 제외한다 **제출**

- •
- 1. 프롬프트에 '반드시 한국어로'를 대문자와 느낌표로 강조해서 다시 적는다
- •
- 2. 출력에 한글이 포함됐는지 코드로 검사하고, 아니면 재요청하거나 번역 단계를 붙인다
- •
- 3. 더 큰 모델로 바꾸면 지시를 더 잘 따르므로 모델을 교체한다
- •
- 4. 영어 기사는 애초에 수집하지 않도록 소스 목록에서 제외한다

기사 수만큼 펼치기

앞의 두 노드는 함수를 바꿔 끼우면 그만이었습니다. 취재는 좀 다릅니다. 기사 한 건마다 본문을 가져오고 언어 모델을 한 번씩 부르니까, 다섯 건을 하나씩 처리하면 다섯 배 걸리거든요.

그런데 몇 건이 될지는 실행해 봐야 압니다. 어떤 날은 세 건이고 어떤 날은 다섯 건이죠. 앞선 수업에서 본 `Send`가 이 문제를 푸는 도구입니다. 조건부 엣지가 노드 이름 대신 `Send` 리스트를 돌려주면 프레임워크가 그 개수만큼 같은 노드를 띄웁니다.

실행 되었습니다.

실습: 취재 노드를 채우고 기사 수만큼 펼치기

실행

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10

11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39

40

41

42

세 번째 줄에 숫자가 있습니다. 바뀐 것은 엡지 한 줄이고 State도 앞뒤 노드도 그대로예요. 섹션 2에서 `drafted` 에만 리듀서를 붙여 둔 것이 여기서 값을 합니다 — 다섯 워커가 각자 `{"drafted": [한 건]}` 을 돌려주면 리듀서가 알아서 합칩니다.

`Send` 의 두 번째 인자를 보세요. 워커가 받는 것은 메인 State가 아니라 `{"item": 기사 하나}` 입니다. 워커는 전체 상황을 모릅니다 — 자기가 맡은 기사만 알고, 다른 기사에 무엇이 있는지, 지금까지 몇 건이 처리됐는지 모릅니다. 제약처럼 보이지만 장점입니다. 워커 하나를 따로 떼어 테스트하기 쉽고, 서로 간섭할 여지도 없습니다.

그리고 본문이 부족하면 빈 리스트를 돌려줍니다. 다섯 건을 뽑았는데 네 건만 통과할 수 있다는 뜻이고, 그 사실이 로그에 남습니다. 그런 날은 네 건만 발행합니다 — 모자란다고 다시 긁어 오지 않아요.

팬아웃 경계

파이프라인에서 일을 여러 벌로 펼칠 수 있는 구간과 없는 구간의 경계입니다. 우리 경우 선별이 끝난 지점이 그 경계이고, 그 앞은 모아서 한 번에, 그 뒤는 건별로 나눕니다.

경계를 잘못 그으면 결과가 조용히 달라집니다. 선별을 기사별로 펼치면 각 워커가 자기 기사만 보고 "중요하다"고 답하게 되고, 다섯 건을 고르는 문제가 아니라 각자 점수를 매기는 문제로 바뀌어요. 섹션 6에서 상대평가를 고른 이유가 통째로 사라지는 겁니다. 프로그램은 잘 돌고 결과만 달라지고요. 반대로 펼칠 수 있는 구간을 안 펼치면 시간만 몇 배로 씁니다.

경계를 찾는 질문은 하나입니다. "이 단계는 다른 항목을 봐야 답할 수 있는가." 봐야 하면 앞쪽(모아서 한 번에), 안 봐도 되면 뒤쪽(건별로 펼침)이에요. 중복 판정·상대평가·상한 적용은 앞쪽이고, 본문 추출·요약·검수는 뒤쪽입니다. 경계는 보통 한 곳이고 거기서 `Send` 가 한 번 등장해요.

펼치면 시간은 줄지만 비용은 줄지 않습니다. 동시에 취재해도 토큰은 그대로 다섯 배예요. 동시 호출이 많아지면 요청 한도에 걸리니 후보가 수십 건인 단계를 무심코 펼치면 안 됩니다. 무엇보다 한 갈래가 실패했을 때 어떻게 할지 정해 두지 않으면 그 건만 조용히 빠진 채 나머지가 합쳐져요. 위 코드에서 본문이 부족한 기사가 빈 리스트로 사라지는 게 그 상황이고, 그래서 빠진 건수를 로그에 남깁니다.

더 해보기 — 취재 단계를 통제해 보기

1. 출력에 한글이 있는지 검사하고 없으면 한 번 더 요청하는 코드를 붙인다

- 접근 힌트: `re.search(r'[가-힣]', out.summary)` 로 판정하고 실패하면 "반드시 한국어로 다시 쓰세요"를 덧붙여 재요청한다. 재시도 횟수를 세어 기록한다

- 성공하면: 재시도가 몇 건 중 몇 건에서 일어나는지 숫자로 알게 된다. 그 숫자가 '프롬프트만으로 될까'라는 질문에 대한 답이다

2. 같은 기사를 temperature 0 과 0.7 로 각각 세 번씩 요약해 차이를 본다

- 접근 힌트: 세 결과의 문장 수·숫자·고유명사가 얼마나 달라지는지 비교한다
- 성공하면: 요약처럼 사실을 옮기는 작업에서 temperature 를 올리는 것이 왜 손해인지 직접 확인할 수 있다. 다양성이 필요한 작업과 아닌 작업의 구분이 생긴다

Chapter09

요약이 원문을 벗어났는지 검사하기

섹션 8에서 만든 요약은 읽기에 매끄럽습니다. 문제는 매끄러운 것과 정확한 것이 다르다는 점이죠. 언어 모델이 지어낸 문장도 똑같이 매끄럽거든요.

사람이 매일 아침 다섯 건을 원문과 대조해 읽으면 확실하겠지만 그러면 자동화의 의미가 없습니다. 기계가 할 수 있는 검사를 만들어야 해요.

가장 먼저 떠오르는 방법부터 해 봅시다. 요약에 나온 숫자가 원문에 실제로 있는지 세어 보는 겁니다. 숫자는 환각이 가장 티 나게 드러나는 자리이고, 문자열 비교라 비용이 0이니깐요.

실행 되었습니다.

실습: 숫자 대조로 환각을 잡아 본다 (키 불필요)

실행

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10

11
12
13
14
15
16
17
18

실제 실행 결과 — 정확한 요약인데 불합격

요약에 등장한 숫자 : ['3', '5', '6000']
원문에 없는 숫자 : ['3', '6000']
판정 : 불합격 — 환각 의심

검사는 정상적으로 동작했습니다. 3 과 6000 은 원문 어디에도 없습니다.
그런데 요약을 다시 읽어 보세요. 틀린 데가 하나도 없습니다.

원문은 just three months after ... raised \$60 million 입니다.

요약은 지난 3개월 전 6000만 달러 규모의 투자 이후 입니다.

- three months → 3개월 — 영어는 숫자를 단어로 썼고 한국어는 아라비아 숫자로 썼습니다.
- \$60 million → 6000만 달러 — 영어의 60 million 을 한국어 단위 체계로 환산했습니다.

번역과 단위 변환을 환각으로 잡은 것입니다. 오탐 두 건이고, 이 검사를 그대로 쓰면 영어 기사를 한국어로 요약할 때마다 거의 매번 불합격이 납니다.

여기서 성급하게 내리기 쉬운 결론이 있습니다 — "숫자 검사는 쓸모없다"입니다. 그렇지 않습니다. 검사가 틀린 것이 아니라 대조 대상이 틀렸습니다. 문자열 "6000" 과 문자열 "60 million" 은 다르지만, 값으로는 같습니다. 문자열을 비교하는 한 이 문제는 풀리지 않습니다.

선택지

정해야 할 것 — 요약이 원문을 벗어나지 않았는지 무엇으로 확인할 것인가?

네 가지가 있고, 아래로 갈수록 정확해지는 대신 비싸집니다.

① 검수하지 않는다

취재 결과를 그대로 발행합니다. 비용이 0이고, 사람이 매일 읽어 볼 수 있는 소규모 발행에 서는 현실적인 선택입니다. 대신 틀린 정보가 나가도 아무도 모릅니다. 뉴스레터는 신뢰가 전부인 물건이라 한 번의 오보가 구독 해지로 이어집니다.

② 규칙 검사 — 숫자·고유명사 대조

요약에 나온 숫자와 이름이 원문에 있는지 문자열로 확인합니다. 비용이 0이고 결과가 결정론적입니다. 같은 언어 안에서는 꽤 잘 듣습니다. 대신 방금 본 것처럼 번역·단위 변환을 전부 오탐합니다. 그리고 숫자가 없는 거짓말('업계 최초로')은 아예 못 잡습니다.

③ LLM 대조 — 요약과 원문을 함께 주고 묻는다 ← 우리가 고른 것

"이 요약이 원문에서 뒷받침되는가"를 언어 모델에게 판정시킵니다. 의미 수준에서 비교하므로 번역과 환산을 올바르게 통과시킵니다. 숫자가 없는 과장도 잡을 수 있습니다. 대신 호출 비용과 지연이 붙고, 판정 자체가 확률적입니다. 검사하는 쪽도 언어 모델이라 그쪽이 틀릴 수 있습니다.

④ 인용 강제 — 근거 문장을 함께 출력하게 한다

요약의 각 문장마다 원문의 근거 구절을 함께 내놓게 하고 그 구절이 원문에 실제로 있는지 문자열로 확인합니다. 판정이 결정론적이면서도 의미를 다룹니다. 근거를 못 대면 그 자리가 바로 환각입니다. 대신 출력이 길어져 비용이 늘고 모델이 근거 구절 자체를 지어낼 수 있어 결국 문자열 확인이 또 필요합니다. 요약의 자유도도 크게 떨어집니다.

우리는 'LLM 대조 — 요약과 원문을 함께 주고 묻는다' 쪽을 골랐습니다. 우리 입력이 영어와 한국어가 섞여 있기 때문입니다. ②는 그 조건에서 구조적으로 오탐을 냅니다. ④는 섹션 8에서 이미 한 번 검토하고 접은 선택지입니다 — 요약이 원문 표현에 묶여 버려서 '세 문장으로 건조하게'라는 우리 요구와 충돌합니다. ①은 신뢰가 전부인 물건에서 위험합니다.

다만 ③을 고르면서 범위를 좁혔습니다. 대조하는 것은 **headline** 과 **summary** 뿐이고 **why** 는 검수하지 않습니다. 섹션 8에서 칸을 나눈 것이 여기서 값을 합니다 — **why** 는 독자 관점의 해석이라 원문에 근거가 있을 수 없고, 대조하면 매번 불합격합니다.

조건이 달라지면 다른 선택지가 맞는 답이 됩니다. 무엇을 포기했는지 알고 고르면 됩니다.

검수

모델이 만든 결과물을 원본과 대조해 통과·불합격을 매기는 별도의 단계입니다. 앞선 수업에서 본 평가자-최적화자 패턴의 평가자에 해당하고, 불합격한 것만 다시 쓰게 하면 루프가 됩니다.

생성 단계에게 "정확하게 써라"라고 부탁하는 것으로는 부족해서 필요해요. 같은 교훈을 여러 번 만났죠. 프롬프트는 요청이지 보장이 아닙니다. 그리고 생성한 주체에게 "네가 쓴 게

맞아?"라고 물으면 대체로 맞다고 답해요. 판정은 생성과 분리된 호출이어야 합니다.

방법은 요약과 원문을 함께 주고 "요약의 각 주장이 원문에서 뒷받침되는가"를 묻는 겁니다. 출력은 {합격 여부, 문제가 된 부분} 처럼 구조화해 받아야 해요. 어디가 문제인지 알아야 다시 쓰게 할 수 있으니까요. 섹션 8의 원칙이 검수 자체의 출력에도 그대로 적용되는 셈입니다.

검수는 비용을 두 배로 만듭니다. 기사마다 생성 한 번 + 검수 한 번이고, 불합격이 나면 재작성까지 세 번이예요. 재시도 횟수에 상한을 두지 않으면 비용이 통제되지 않습니다. 그리고 검수도 언어 모델이라 틀립니다. 멀쩡한 요약을 떨어뜨리기도 하고 진짜 환각을 통과시키기도 해요. 검수를 붙였다고 정확성이 보장되는 게 아니라 틀릴 확률이 줄어드는 거고, 그 차이를 혼동하면 검수를 믿고 다른 안전장치를 빼게 됩니다.

이 결정을 검증하는 방법

확인할 것 — LLM 검수가 실제로 환각을 걸러 내고 있고, 멀쩡한 요약을 부당하게 떨어뜨리지 않는다.

아래 신호로 확인합니다.

- **불합격률(verify_fail)** — 실행마다 검수에서 떨어진 건수를 기록합니다. 0이 계속되면 검수가 통과 도장만 찍고 있을 가능성이 있고, 너무 높으면 기준이 과하거나 취재 프롬프트가 잘못된 것입니다.
- **재작성 후 합격률** — 불합격한 건을 다시 써서 통과한 비율입니다. 재작성해도 계속 떨어지면 원인이 요약이 아니라 본문 추출에 있을 수 있습니다.
- **일부러 망가뜨린 입력의 적발 여부** — 요약에 없는 숫자를 손으로 넣은 가짜 요약을 가끔 흘려보냅니다. 그것이 통과하면 검수가 작동하지 않는 것입니다.

세 번째가 중요합니다. 검수처럼 평소에 아무 일도 안 하는 장치는 고장 나도 티가 나지 않습니다. 섹션 5의 '조용한 실패'가 검사 장치 자체에서도 일어나는 것입니다. 일부러 틀린 입력을 넣어 잡히는지 확인하는 것을 습관으로 삼아야 합니다.

틀렸을 때의 증상 — 검수가 무력해졌을 때의 증상은 아무 증상도 없는 것입니다. 불합격이 0건이고 모든 것이 매끄럽게 발행됩니다. 그래서 `verify_fail` 이 0이라는 것은 좋은 소식이 아니라 확인이 필요하다는 신호로 읽어야 합니다.

객관식 퀴즈

Q. 숫자 대조 검사가 `three months → 3개월`, `\$60 million → 6000만 달러`를 환각으로 잡아냈습니다. 이 결과에 대한 해석으로 가장 적절한 것은?

- 1. 검사 코드에 버그가 있으므로 정규식을 고쳐야 한다
- 2. 검사는 정상 동작했지만 문자열 일치라는 대조 방식이 번역·단위 변환을 다룰 수 없는

것이므로, 의미 수준의 대조로 바뀌어야 한다

- 3. 요약은 영어로 쓰게 하면 해결되므로 한국어 요구를 포기한다
- 4. 숫자가 포함된 기사는 검수 대상에서 제외하면 된다

• •

1. 검사 코드에 버그가 있으므로 정규식을 고쳐야 한다

• •

2. 검사는 정상 동작했지만 문자열 일치라는 대조 방식이 번역·단위 변환을 다룰 수 없는 것이므로, 의미 수준의 대조로 바뀌어야 한다

• •

3. 요약은 영어로 쓰게 하면 해결되므로 한국어 요구를 포기한다

• •

4. 숫자가 포함된 기사는 검수 대상에서 제외하면 된다

주관식 퀴즈

Q. 우리는 검수 대상을 `headline` 과 `summary` 로 한정하고 `why` 는 검수하지 않기로 했습니다. 그런데 `why` 야말로 독자가 읽고 판단의 근거로 삼는 부분입니다. 검사하지 않는 칸을 그대로 발행하는 것이 위험하지 않을까요? 위험하다면 그 위험을 다른 방법으로 어떻게 줄일 수 있을지 제안해 보세요.

0/300다음

검수 노트 작성하기

검수는 취재와 달리 펼치지 않습니다. 기사마다 판정이 필요하니 펼칠 수도 있지만, 다섯 건이면 한 노트 안에서 순서대로 도는 편이 단순하고 동시 호출도 줄어드니까요. 펼칠 수 있다고 늘 펼쳐야 하는 건 아닙니다.

정할 것은 그다음입니다. 불합격한 요약을 어떻게 할 것인가.

세 가지가 가능해요. ① 그 자리에서 다시 쓰게 한다 — 지적을 붙여 취재를 한 번 더 부릅니다. 값은 건지지만 비용이 늘고 재시도 이력이 노트 안에 숨죠. ② 취재 노트로 되돌아가는 엷지를 놓는다 — 그래프에 그려져 보기 좋지만 다섯 건 중 한 건 때문에 스텝이 통째로 한 번 더 돕니다. ③ 그냥 버리고 로그에 남긴다 — 가장 단순하고, 대신 건수가 줄어듭니다.

우리는 ③을 고릅니다. 발행 규칙을 단순하게 잡았기 때문이에요 — 검수를 통과한 만큼 그대로 보냅니다. 다섯 건을 채우려고 다시 쓰거나 다시 긁어 오지 않습니다. 본문이 없어 취재

에서 빠진 건이든 검수에서 떨어진 건이든 결과는 똑같이 '그날은 네 건'이고, 그 사실이 로그에 남으면 충분합니다.

되돌아가는 경로를 붙이려면 '언제까지 다시 시도할 것인가'를 정해야 하고, 거기서부터 재시도 횟수·비용 상한·중복 누적 같은 것이 줄줄이 따라옵니다. 그건 이 파이프라인이 감당할 복잡도가 아니에요.

실행 되었습니다.

실습: 검수 노드를 채우고 다시 돌리기

실행

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23
24
25
26
27
28
29
30
31
32
33
34
35
36

네 번째 줄까지 찾습니다. 그런데 여기서 걸림돌이 하나 나옵니다. `drafted` 에는 리듀서가 붙어 있습니다(섹션 2).

취재 워커들의 결과를 합치려고 붙인 것인데, 검수는 반대로 줄이는 일을 합니다.

`operator.add` 가 붙은 키에 통과분만 돌려주면 줄어들지 않고 늘어납니다.

그래서 통과분을 `verified` 라는 새 키에 담았습니다. 합치는 키와 줄이는 키를 나눈 것입니다. State에 칸이 하나 늘었지만, 섹션 2에서 정한 규칙 — 노드는 자기가 바꾼 키만 돌려준다 — 덕분에 다른 노드는 하나도 고치지 않았습니다.

더 해보기 — 검수를 직접 붙여 보기

1. LLM 검수를 구현해 방금 그 요약이 통과하는지 확인한다

- 접근 힌트: `{ok: bool, problems: list[str]}` 스키마로 받고 요약과 원문을 함께 넣어 "요약의 각 주장이 원문에서 뒷받침되는가"를 묻는다
- 성공하면: 숫자 대조가 오탐했던 `6000만 달러` 를 LLM 은 통과시키는지 직접 확인할 수 있다. 통과한다면 ③을 고른 판단이 옳았다는 근거가 된다

2. 요약에 없는 숫자를 손으로 끼워 넣은 가짜 요약을 만들어 검수가 잡는지 본다

- 접근 힌트: `기업 가치는 약 5억 달러` 를 `약 50억 달러` 로 바꿔 넣고 돌린다

- 성공하면: 검수 장치가 실제로 작동하는지 확인하는 습관이 생긴다. 위 검증 블록의 세 번째 지표가 바로 이것이다

Chapter10

마지막 노드 — 발행

네 개를 채웠고 **publish** 하나가 남았습니다. 이 노드는 앞의 넷과 성질이 달라요. 밖으로 나갑니다. 수집을 두 번 해도, 요약을 다시 써도 세상은 아무 일 없지만 발행은 보내는 순간 남의 화면에 뜨니까요.

건수 규칙은 단순하게 갑니다. 검수를 통과한 만큼 그대로 보냅니다. 한 건도 없는 날에는 "오늘은 조용합니다" 한 줄만 보냅니다. 아무것도 안 보내면 파이프라인이 죽은 것과 구분이 안 되니까요.

이제는 만든 걸 실제로 받아 보는 거예요. 지금까지는 노트북 출력으로만 봤죠. 매일 아침 휴대전화에 알림이 터야 비로소 만들었다는 실감이 납니다. 그래서 5분 안에 끝나는 경로를 고릅니다.

선택지

정해야 할 것 — 완성된 브리핑을 어디로 보낼 것인가?

네 가지가 있고, 위로 갈수록 제대로 된 뉴스레터에 가깝지만 시작하는 데 오래 걸립니다.

① 이메일

SMTP 서버나 발송 서비스(SendGrid·Resend 등)로 HTML 메일을 보냅니다. 뉴스레터의 표준 경로입니다. 구독자 관리·수신거부·오픈율 측정이 갖춰져 있고 받는 사람이 앱을 깔 필요도 없습니다. 대신 시작이 가장 오래 걸립니다. 발송 도메인 인증(SPF·DKIM) 없이 보내면 스팸함으로 갑니다.

② Slack 앱 / 웹훅

워크스페이스에 들어오는 웹훅을 만들고 JSON을 보냅니다. 팀이 이미 Slack을 쓴다면 아무도 새로 가입하지 않아도 됩니다. 대신 관리자 승인이 필요할 수 있고, 연습용 워크스페이스를 새로 만드는 것은 번거롭습니다.

③ Discord 웹훅 ← 우리가 고른 것

서버의 채널에서 웹훅 URL 하나를 발급받아 그 주소로 JSON을 **POST** 합니다. 가장 빨리 확인할 수 있습니다. 계정만 있으면 서버를 혼자 만들 수 있고, 승인 절차가 없으며, URL 하나로 끝납니다. **embed** 라는 구조화된 카드 형식이 있어 제목·요약·링크·출처를 보기 좋게 나눌

수 있습니다. 대신 구독자 관리라는 개념이 없습니다. 누가 읽었는지 알 수 없고, 디스코드를 쓰지 않는 사람에게는 보낼 수 없습니다. 진짜 뉴스레터로 키우려면 결국 ①로 옮겨야 합니다.

④ 정적 페이지로 발행

결과를 마크다운·HTML로 만들어 GitHub Pages 같은 곳에 올립니다. 링크 하나로 공유되고 과거 회차가 그대로 쌓입니다. 대신 알림이 없습니다. 사람이 찾아와야 하므로 '매일 아침 받아 본다'는 경험이 안 만들어집니다.

우리는 Discord 웹훅을 골랐습니다. 오늘 안에 결과를 손에 쥐는 것이 이 섹션의 목적이기 때문입니다. ③은 계정 하나로 5분이면 끝나고, 그 5분이 이후 모든 섹션의 확인 수단이 됩니다. 자동 실행을 붙인 뒤 "오늘 아침에 돌아왔나"를 알림 하나로 확인할 수 있다는 것도 큼니다.

다만 ③이 '이메일보다 낫다'는 뜻은 아닙니다. 실제 독자에게 보내는 뉴스레터라면 ①이 맞습니다. ③으로 시작하는 이유는 발행 단계의 구조 — 무엇을 어떤 모양으로 내보내고, 한도를 어떻게 다루고, 실패하면 어떻게 아는지 — 가 경로와 무관하게 같기 때문입니다. 나중에 ①로 바꿀 때 고치는 것은 `publish` 노드 하나뿐입니다.

조건이 달라지면 다른 선택지가 맞는 답이 됩니다. 무엇을 포기했는지 알고 고르면 됩니다.

웹훅 주소 받기

여기부터는 읽지만 말고 따라 해 보세요. 디스코드 계정만 있으면 3분이면 됩니다.

1. 디스코드를 열고 왼쪽 아래 `+` 를 눌러 서버를 하나 만듭니다. "직접 만들기" → "나와 친구들을 위한 서버" 를 고르면 됩니다. 이름은 아무거나 좋습니다.
2. 서버 안에 채널을 하나 만듭니다(예: `#브리핑`). 기본으로 생긴 `#일반` 을 써도 됩니다.
3. 채널 이름 옆 톱니바퀴(채널 편집) → 왼쪽 메뉴에서 연동 → 웹훅 → 새 웹훅.
4. 이름을 '편집실' 정도로 바꾸고 웹훅 URL 복사 를 누릅니다. `https://discord.com/api/webhooks/숫자/긴문자열` 형태입니다.
5. 그 값을 섹션 6의 키 설정 셀에 붙여 넣고 그 셀을 다시 실행합니다.

```
DISCORD_WEBHOOK_URL="https://discord.com/api/webhooks/..."
```

터미널을 쓸 수 있다면 셀에서 넣어도 됩니다. 키 설정 셀의 그 줄을 비워 두면 셀 값을 그대로 씁니다.

```
export DISCORD_WEBHOOK_URL="https://discord.com/api/webhooks/..." # macOS · Linux
```

어느 쪽이든 **노트북에 주소를 적은 채로 공유하면 안 됩니다**. 웹훅 URL에는 인증이 따로 없어서,

주소를 아는 사람은 누구나 그 채널에 글을 쓸 수 있어요. 아이디와 비밀번호가 아니라 주소 자체가 열쇠입니다. 새어 나갔다면 디스코드에서 그 웹훅을 지우고 새로 만드세요.

한 줄 보내 보기

구조화는 나중 일이고, 일단 도착하는지부터 확인합니다.

실행 되었습니다.

실습: 웹훅으로 첫 메시지 보내기

실행

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9

무엇을 보게 되나

성공하면 **204** 와 **(본문 없음)** 이 찍힙니다. 디스코드는 성공했을 때 돌려줄 내용이 없다는 뜻으로 204를 씁니다. 200이 아니라고 놀라지 마세요. 채널을 보면 방금 그 문장이 올라와 있습니다 — 파이프라인의 출구가 열린 것입니다.

실패했다면 숫자로 원인을 좁힐 수 있습니다. 401 · 404 는 주소가 틀렸거나 웹훅이 삭제된 것, 429 는 요청 한도(응답 본문에 몇 초 뒤 재시도하라는 값이 들어 있습니다), 400 은 보낸 JSON이 규격에 안 맞는 경우입니다.

embed 로 구조 주기

`content` 는 한 덩어리 텍스트라 다섯 건을 이어 붙이면 벽처럼 보입니다.

디스코드의 `embed` 는 제목·본문·링크·출처를 나눠 담는 카드 형식입니다. 섹션 9에서 출력을 세 칸으로 나눈 것이 여기서 값을 합니다. `headline` 은 카드 제목으로, `summary` 는 본문으로, `why` 는 강조 줄로 작성하겠습니다.

한도가 있습니다. 메시지 하나에 embed 10개, 제목 256자, 본문 4096자, 그리고 전부 합쳐 6000자입니다. 넘으면 400이 돌아오고 아무것도 발행되지 않습니다. 그래서 보내기 전에 잘라 두어야 합니다.

아래 코드에는 `dry_run` 이 있습니다. 보내지 않고 무엇을 보낼지만 출력합니다. 매일 도는 물건을 만들 때 이 스위치는 선택이 아니라 필수입니다 — 프롬프트를 고칠 때마다 채널에 시험 발행이 쌓이면 안 되기 때문입니다.

실행 되었습니다.

실습: embed 로 만들고 dry-run 으로 확인하기 (키 불필요)

실행

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15
- 16

17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45

46
47
48
49
50
51
52
53
54

실제 실행 결과 — dry-run 출력

```
[dry-run] embed 3개 · 590자 - 보내지 않음
{
  "title": "1. 오픈AI, GPT-6 아스트라 공개",
  "description": "오픈AI가 차세대 모델을 공개했습니다. 컨텍스트 한도가 두 배로 늘었습니다.

💡 지금 쓰는 프롬프트를 그대로 옮길 수 있는지부터 확인해야 합니다.",
  "url": "https://example.com/a",
  "color": 749175,
  "footer": {
    "text": "전자신문 · 2시간 전"
  }
}
```

embed는 3개입니다 — 머리말 한 장 + 기사 두 장. 기사가 다섯 건이면 여섯 장이 되고, 한도인 10장까지는 여유가 있습니다.

첫 기사 카드를 보면 섹션 9에서 나눈 세 칸이 각자 다른 자리에 들어갔습니다. `headline` 은 `title` 로, `summary` 는 `description` 으로, `why` 는 그 아래 💡 표시와 함께 굵게 붙었습니다. 한 덩어리로 받았다면 이렇게 배치할 수 없었습니다.

`color` 가 `749175` 로 찍힌 것은 `0x0B6E77` 을 10진수로 보여 준 것뿐입니다. 디스코드는 색을 정수로 받습니다.

`dry_run=True` 를 `False` 로 바꾸면 실제로 채널에 올라갑니다. 지금 한 번 해 보세요.

발행 노드 작성하기

남은 건 끼우는 일뿐입니다. `dry_run` 을 State가 아니라 설정으로 받는 점을 보세요.
실행할 때마다 달라지는 값이 아니라 이 실행이 어떤 실행인지를 정하는 값이거든요.
실행 되었습니다.

실습: 발행 노드를 채우고 전체를 한 번 돌리기

실행

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19

다섯 줄이 전부 0이 아닌 숫자로 찻습니다. 여기까지가 파이프라인 완성입니다.

건수가 모자라도 그대로 나가고, 0건이면 `build_embeds` 가 "오늘은 조용합니다" 한 장만 만들어 보냅니다. 조건 분기 하나로 끝나는 이유는 되돌아가는 경로를 만들지 않았기 때문이에요. 재수집이나 기준 완화를 붙이면 그때부터 '언제까지 다시 시도할 것인가'를 정해야 하고, 그건 이 파이프라인이 감당할 복잡도가 아닙니다.

`DRY_RUN` 을 환경변수로 읽고 기본값을 `"1"` (보내지 않음) 으로 두었습니다. 실수로 발행되는 쪽보다 실수로 안 보내는 쪽이 덜 나쁘니까요. 실제로 보내려면 `os.environ["DRY_RUN"] = "0"` 을 먼저 실행하세요.

`publish` 가 `verified` 를 꺼내 필요한 칸만 골라 새 딕셔너리를 만드는 것도 보세요. State에는 `body` 처럼 발행에 쓰지 않는 칸이 들어 있습니다. 발행 단계에 그대로 넘기면 디스코드 한도(6000자)를 넘기기 쉽고, 무엇을 내보내는지도 흐려집니다.

웹훅

미리 받아 둔 주소로 HTTP 요청을 보내면 그쪽 시스템에 무언가가 일어나는 방식입니다. 디스코드 웹훅은 그 주소로 JSON을 `POST` 하면 채널에 메시지가 떠요. SDK도 인증 절차도 없고, 주소와 `requests` 하나면 끝납니다.

연동을 가장 값싸게 만드는 방법이라 널리 쓰입니다. 상대 시스템의 API를 배우고 토큰을 발급받고 권한을 설정하는 과정이 전부 생략되거든요. 알림·배포 통지·모니터링처럼 한 방향으로 알리기만 하면 되는 연동에 잘 맞습니다.

받는 쪽이 발급한 URL 안에 식별자와 비밀 토큰이 함께 들어 있습니다. 그래서 별도 인증 해더 없이 그 주소로 보내는 것만으로 인증이 끝나요. 보내는 쪽은 정해진 JSON 모양(우리 경우 `content` 또는 `embeds`)만 맞추면 됩니다.

그래서 URL이 곧 비밀번호입니다. 저장소에 올리거나 노트북 셀에 적어 공유하면 누구나 그 채널에 글을 쓸 수 있고, 되돌리는 방법은 웹훅을 삭제하고 다시 만드는 것뿐이에요. 그리고 웹훅에는 중복 방지가 없습니다. 같은 내용을 두 번 보내면 두 번 뜨니, 재시도를 붙일 때 '보냈는지 모르겠으니 한 번 더'가 바로 중복 발송이 됩니다. 요청 한도(429)도 있어서 여러 건을 쪼개 보낸다면 간격을 두어야 하고요.

객관식 퀴즈

Q. 디스코드 웹훅으로 메시지를 보냈더니 `204`가 돌아왔고 응답 본문은 비어 있었습니다. 이 결과에 대한 해석으로 가장 적절한 것은?

- 1. 본문이 비어 있으므로 전송이 실패했고 재시도해야 한다
 - 2. 204는 '성공했고 돌려줄 내용이 없다'는 뜻이므로 정상이며, 채널을 확인하면 메시지가 올라와 있다
 - 3. 204는 요청 한도에 걸렸다는 뜻이므로 잠시 뒤 다시 보내야 한다
 - 4. 웹훅은 항상 200을 돌려주므로 204가 나왔다면 주소가 잘못된 것이다
- 제출
- •
 - 1. 본문이 비어 있으므로 전송이 실패했고 재시도해야 한다
 - •

2. 204는 '성공했고 돌려줄 내용이 없다'는 뜻이므로 정상이며, 채널을 확인하면 메시지가 올라와 있다

• •

3. 204는 요청 한도에 걸렸다는 뜻이므로 잠시 뒤 다시 보내야 한다

• •

4. 웹훅은 항상 200을 돌려주므로 204가 나왔다면 주소가 잘못된 것이다

더 해보기 — 발행을 내 손에 맞추기

1. `dry_run=False` 로 실제 발행해 보고, 색과 문구를 바꿔 다시 보낸다

- 접근 힌트: `COLORS` 의 토픽별 색을 자기 취향으로 바꾸고 머리말 문구도 고쳐 본다. 채널에서 어떻게 보이는지 확인한다
- 성공하면: 설정 몇 줄이 받아 보는 경험을 얼마나 바꾸는지 직접 알게 된다. 그리고 이 화면이 앞으로 모든 섹션의 확인 수단이 된다

2. 기사를 일부러 20건 넣어 한도 방어가 동작하는지 확인한다

- 접근 힌트: `ARTICLES` 를 복제해 20건으로 늘리고 dry-run 으로 embed 개수를 센다. `EMBED_MAX` 를 30으로 바꿔 실제 발행하면 어떤 응답이 오는지도 본다
- 성공하면: 한도를 넘기면 아무것도 발행되지 않는다는 것을 직접 보게 된다. 보내기 전에 자르는 코드가 왜 필요한지 분명해진다

웹훅과 Discord 참고 자료

- [Discord 개발자 문서 — Webhook Resource](#) — 웹훅으로 보낼 수 있는 필드와 응답 코드가 정리돼 있다. 우리가 쓴 `username · embeds` 가 여기 있고, 중복 방지 기능이 없다는 것도 확인할 수 있다
- [Discord 개발자 문서 — Message Embed 구조와 한도](#) — 제목 256자·본문 4096자·합계 6000자·embed 10개라는 한도가 적혀 있다. 코드의 상수들이 어디서 왔는지 확인할 수 있다
- [MDN — HTTP 204 No Content](#) — 성공인데 본문이 없는 응답이 왜 필요한지 짧게 정리돼 있다. 방금 본 204를 실패로 오해하지 않기 위한 배경이다

Chapter11

자동 실행에 필요한 것

섹션 10에서 알림을 받아 봤다면 마지막 한 걸음이 남았습니다. 노트북을 열지 않아도 매일 아침 7시 30분에 저절로 도는 것이죠.

필요한 건 세 가지입니다. 정해진 시각에 깨울 것, API 키를 안전하게 넘길 것, 돌았는지 확인할 경로. 세 번째가 가장 자주 빠집니다.

GitHub Actions를 쓰겠습니다. 저장소만 있으면 되고, 공개 저장소는 무료이고, 실행 로그가 웹에 그대로 남거든요.

저장소 구성하기

노트북(ipynb)은 자동 실행에 맞지 않습니다. 셀을 순서대로 돌려 주는 사람이 없으니까요. 파이썬 파일로 떨어뜨리는 게 먼저입니다.

```
my-newsletter/
├── graph.py          # 노트북에서 만든 것 전부 - State · 노드 다섯 개 · build() · run()
├── run.py            # graph.py 를 불러 한 번 실행한다 - 열 줄 남짓
├── requirements.txt  # feedparser · requests · trafilatura · openai · langgraph
├── pydantic
├── store/            # 실행 기록이 쌓이는 곳 (섹션 12)
└── .github/workflows/daily.yml
```

노트북 셀은 거의 전부 `graph.py` 한 파일로 들어갑니다. 어느 섹션이 어디로 가는지 정리하면 이렇습니다.

노트북	<code>graph.py</code> 에 들어가는 것
섹션 2	<code>Brief</code> · <code>build()</code> · <code>INIT</code> · <code>run()</code>
섹션 5	<code>SOURCES</code> · <code>strip_tags</code> · <code>published_at</code> · <code>collect</code>
섹션 7	<code>Pick</code> · <code>Shortlist</code> · <code>CRITERIA</code> · <code>ask_picks</code> · <code>select</code>
섹션 8	<code>Draft</code> · <code>extract_body</code> · <code>draft</code> · <code>fan_report</code> · <code>report</code> · <code>build()</code> 재정의
섹션 9	<code>Verdict</code> · <code>check</code> · <code>verify</code> · <code>Brief</code> 재정의
섹션 10	<code>build_embeds</code> · <code>send</code> · <code>make_lead</code> · <code>publish</code>
섹션 12	<code>run()</code> 재정의 — 실행 기록 한 줄 남기기

옮길 때 세 가지를 손봅니다.

`! pip install` 줄은 `requirements.txt` 로 갑니다. 파이썬 파일에서는 `!` 를 쓸 수 없어요.

같은 이름을 두 번 정의한 것은 나중 것만 남깁니다. 노트북에서는 `build()` 와 `Brief` 를 중간에 다시 정의했죠. 파일에서는 최종 버전 하나만 두면 됩니다. 위에서 아래로 읽었을 때

정의가 먼저 나오고 쓰는 곳이 뒤에 오도록 순서만 맞추세요.

실습 셀에 있는 스키마도 빠뜨리지 마세요. 섹션 7과 8은 코드 셀이 둘씩입니다.

앞 셀(실습)에서 `Pick · Shortlist · Draft · SYS` 를 정의하고 뒤 셀(노드)이 그걸 씁니다.

노드 셀만 옮기면 `NameError: name 'Shortlist' is not defined` 로 멈춰요.

앞 셀에서는 **클래스와 상수만** 가져오고, `titles` 를 훑는 실습용 반복문은 노트북에 두면 됩니다.

`run.py` 는 `graph.py` 를 불러 한 번 실행하는 것이 전부입니다.

```
# run.py
import os, sys
from graph import run# graph.py 에 있는 그 run()

if __name__=="__main__":
    if "--dry-run" in sys.argv:
        os.environ["DRY_RUN"]="1"# 섹션 10의 스위치
        out= run()
    for line in out["log"]:
        print(line)# 이 출력이 Actions 로그에 그대로 남는다
```

진입점을 따로 둔 이유가 있습니다. `graph.py` 는 불러오기만 해도 안전해야 해요.

거기에 `run()` 호출을 적어 두면 `import graph` 하는 순간 브리핑이 발행됩니다.

실행은 `if __name__ == "__main__"` 아래에서만 일어나야 합니다.

그리고 마지막 두 줄 — 화면에 찍는 것이 곧 로그입니다.

키를 Secrets 에 등록하기

`OPENAI_API_KEY` 와 `DISCORD_WEBHOOK_URL` 을 코드에 적으면 안 됩니다.

공개 저장소라면 몇 분 안에 긁어가는 봇들이 있어요. 넣는 자리가 따로 있습니다.

- 저장소 페이지에서 Settings → 왼쪽 메뉴 Secrets and variables → Actions.
- New repository secret 을 눌러 두 개를 등록합니다.
 - 이름 `OPENAI_API_KEY` , 값은 발급받은 키
 - 이름 `DISCORD_WEBHOOK_URL` , 값은 섹션 10에서 복사한 주소
- 등록하면 다시 볼 수 없습니다. 확인하는 화면이 없고 덮어쓰기만 됩니다. 불편이 아니라 설계입니다 — 화면에 보이는 순간 유출 경로가 하나 생깁니다.
- 저장소에 올릴 코드에서는 **섹션 6에서 하드코딩한 두 줄을 반드시 비우세요.** 공개 저장소에 키가 한 번 올라가면 커밋을 지워도 이력에 남습니다.

워크플로에서는 `{{ secrets.OPENAI_API_KEY }}` 로 꺼내 씁니다. 로그에 실수로 찍더라도 GitHub가 * 로 가려 주지만**, 값을 변형해 찍으면(앞 10자만 잘라서 등) 가려지지 않습니다.

워크플로 파일

`.github/workflows/daily.yml` 을 만들고 아래를 붙여 넣습니다.
경로가 정확해야 GitHub가 알아봅니다.

```
name: 매일 브리핑

on:
  schedule:
    - cron: "30 22 * * *" # ① UTC 22:30 = KST 07:30
  workflow_dispatch: # ② 손으로도 돌릴 수 있게
  inputs:
    dry_run:
      description: "발행하지 않고 로그만 남기기"
      type: boolean
      default: false

jobs:
  publish:
    runs-on: ubuntu-latest
    timeout-minutes: 20 # ③ 멈춰 있는 실행을 20분에서 끊는다

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
        cache: pip

      - run: pip install -r requirements.txt

      - name: 브리핑 생성·발행
        run: python run.py ${ inputs.dry_run && '--dry-run' || '' }
        env: # ④ 비밀값은 여기로만 들어간다
          OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY }
          DISCORD_WEBHOOK_URL: ${ secrets.DISCORD_WEBHOOK_URL }
          TZ: Asia/Seoul

      - name: 실행 기록 보관 # ⑤ 로그 말고 파일로 남은 것도 챙긴다
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: run-${ github.run_number }
          path: store/
          retention-days: 30
```

다섯 군데에 결정이 들어 있습니다.

① `cron` 은 UTC 기준입니다. 한국시간 07:30에 돌리려면 22:30을 적어야 하고 전날 날씨가 됩니다. 그리고 정시 실행은 보장이 아니라 최선의 노력이라 몇 분에서 한 시간까지 밀립니다.

② `workflow_dispatch` 가 없으면 고칠 수 없습니다. 넣어 두면 Actions 탭에 Run workflow 버튼과 `dry_run` 체크박스가 생깁니다. 자동화의 첫 단추는 손으로 돌릴 수 있게 만드는 것입니다.

③ `timeout-minutes` 가 없으면 응답 없는 요청에 매달려 무료 사용량을 갉아먹습니다.

④ 비밀값은 `env` 로만 들어갑니다. 코드에도 설정 파일에도 적지 않습니다.

⑤ `upload-artifact` 는 파일로 남은 것을 챙깁니다. 실행 환경은 끝나면 사라지므로 `store/` 에 쌓은 기록도 함께 사라집니다. `if: always()` 를 붙여 실패했을 때도 가져옵니다

—
실패한 회차의 기록이야말로 가장 보고 싶은 것이기 때문입니다.

손으로 먼저 실행하기

파일 세 개를 커밋하고 푸시했다면 저장소 상단의 Actions 탭으로 갑니다.

1. 왼쪽에서 매일 브리핑 을 고릅니다.
2. 오른쪽 Run workflow → `dry_run` 을 체크한 채 실행합니다.
3. 잠시 뒤 생긴 실행을 눌러 브리핑 생성·발행 단계를 펼치면 `run.py` 가 찍은 다섯 줄이 그대로 보입니다.
4. 로그가 정상이면 `dry_run` 을 끄고 다시 실행합니다. 디스코드 채널에 브리핑이 올라오면 끝입니다.

여기까지 되면 내일 아침부터는 아무것도 하지 않아도 알림이 옵니다.

스케줄 실행은 기본 브랜치에 워크플로 파일이 있어야 동작합니다.
다른 브랜치에서 시험하는 중이라면 `schedule` 은 울리지 않습니다.
그리고 공개 저장소라도 60일 동안 커밋이 없으면 스케줄이 자동으로 멈춥니다.
멈추면 GitHub가 메일로 알려 주고, Actions 탭에서 다시 켤 수 있습니다.

실행됐는지 확인하는 세 가지 방법

자동화에서 가장 흔한 사고는 3주 전부터 멈춰 있었는데 아무도 몰랐던 겁니다.
막는 방법은 세 겹이에요.

첫째, 실패하면 GitHub가 메일을 보냅니다. 기본으로 켜져 있어 '요란하게 죽는 고장'은 잡힙니다.

둘째, 조용한 날에도 무언가를 보냅니다. 섹션 10에서 0건일 때 '오늘은 조용합니다'를 보내게 한 것이 여기서 두 번째 역할을 합니다. 소식이 없어 조용한 것과 파이프라인이 죽어서 조용한 것은

겉보기에 같은데, 전자는 메시지가 오고 후자는 오지 않습니다.

셋째, 실행 목록을 가끔 봅니다. Actions 탭의 초록·빨강 점만 훑어도 보입니다.

둘째가 가장 값싸고 잘 듭니다. 성공 여부가 아니라 '오늘 실행됐는가'를 받는 사람이 직접 확인하게 되기 때문입니다.

스케줄 실행과 관측

정해진 시각에 사람 없이 실행되는 구조입니다. 크론 표현식으로 시각을 정하고, 실행 환경이

매번 새로 만들어지며, 결과는 로그와 우리가 남긴 기록으로만 확인됩니다.

매일 해야 하는 일을 사람이 기억하는 건 오래 못 갑니다. 그런데 자동화의 진짜 효과는 일을 덜어 주는 게 아니라 일을 보이지 않게 만드는 것이고, 보이지 않는 일은 멈춰도 티가 안 나요. 그래서 자동화는 관측과 짝으로만 의미가 있습니다. 섹션 12의 지표가 그 짝이고요.

스케줄러가 깨끗한 환경을 만들고, 코드를 내려받고, 비밀값을 주입한 뒤 실행합니다.

끝나면 환경은 통째로 사라지니 남겨야 할 것을 명시적으로 밖으로 빼야 해요.

artifact 업로드와 디스코드 메시지가 그 통로입니다.

주의할 건 셋입니다. 정시 실행은 최선의 노력일 뿐이고, 저장소가 조용하면 스케줄이 꺼집니다.

환경이 매번 깨끗하니 이전 실행이 만든 파일이 남아 있다고 가정하면 안 됩니다.

무료 사용량에도 한도가 있으니 `timeout-minutes` 는 꼭 걸어 두세요.

객관식 퀴즈

Q. 워크플로에 `workflow\_dispatch`를 함께 넣는 이유로 가장 적절한 것은?

- 1. `schedule` 만으로는 워크플로 파일이 인식되지 않기 때문이다
- 2. 손으로 즉시 실행할 수 있어야 고칠 수 있기 때문이다 — 내일 아침을 기다리지 않고 dry-run 으로 확인한 뒤 실제 발행으로 넘어갈 수 있다
- 3. `workflow\_dispatch` 가 있어야 secrets 를 읽을 수 있기 때문이다
- 4. 수동 실행은 무료 사용량에서 차감되지 않기 때문이다 **제출**

• •

1. `schedule` 만으로는 워크플로 파일이 인식되지 않기 때문이다

• •

2. 손으로 즉시 실행할 수 있어야 고칠 수 있기 때문이다 — 내일 아침을 기다리지 않고 dry-run 으로 확인한 뒤 실제 발행으로 넘어갈 수 있다

• •

3. `workflow\_dispatch` 가 있어야 secrets 를 읽을 수 있기 때문이다

• •

4. 수동 실행은 무료 사용량에서 차감되지 않기 때문이다

더 해보기 — 자동 실행을 내 것으로

1. `cron` 을 자기 생활에 맞추고 5분 뒤로 한 번 시험해 본다

- 접근 힌트: UTC 기준이라는 점에 주의해 계산한다. 시험이 끝나면 원래 시각으로 되돌리는 것을 잊지 말 것
- 성공하면: 크론 표현식이 UTC라는 사실이 몸에 남는다. 그리고 스케줄이 정확히 그 시각에 오지 않는다는 것도 직접 보게 된다

2. 일부러 실패하는 실행을 만들어 알림이 오는지 확인한다

- 접근 힌트: secrets 이름을 한 글자 바꿔 키를 못 찾게 하고 돌린다. 메일이 오는지, Actions 탭에 빨간 점이 찍히는지 본다
- 성공하면: 안전장치는 작동하는 것을 한 번 봐 두어야 믿을 수 있다. 섹션 9에서 검수 장치에 일부러 틀린 입력을 넣어 본 것과 같은 습관이다

자동 실행 참고 자료

- GitHub Actions — 워크플로를 트리거하는 이벤트
트 — `schedule` 과 `workflow_dispatch` 의 정확한 동작이 정리돼 있다. 스케줄이 지연될 수 있다는 설명과 60일 비활성 시 중단 규칙을 확인해 두면 좋다
- crontab.guru — 크론 표현식 해석기 — `30 22 * * *` 가 언제인지 문장으로 풀어 준다. UTC로 계산하는 습관을 들일 때 옆에 띄워 두면 실수가 줄어든다

Chapter12

내린 결정을 재 보기

돌아보면 열 개가 넘는 갈림길에서 한쪽을 골랐습니다. RSS를 쓰기로 했고, 관문을 세 개로 정했고,

본문 기준을 600자로 잡았고, 예선 묶음을 40건으로 나눴고, 발행 목표를 다섯 건으로 뒀죠.

그런데 근거로 삼은 건 대부분 한 번의 측정이었습니다. 소스당 기사 몇 건, 후보 열두 건, 실행 한 번.

그것만으로 "이 구조가 맞다"고 말하기는 어렵습니다. 설계 결정이 취향과 다른 점이 여기에요.

재어 볼 수 있어야 합니다.

실행마다 한 줄씩 남기기

방법은 단순합니다. 실행이 끝날 때마다 `store/metrics.jsonl` 에 JSON 한 줄을 덧붙이면 돼요.

데이터베이스도 대시보드도 모니터링 도구도 필요 없습니다. 파일 하나에 append 입니다.

지금까지 써 온 `run()` 을 그 일까지 하도록 다시 정의합니다.

실행 되었습니다.

실습: 실행 기록을 한 줄 남기고 열어 보기

실행

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15

16
17
18
19
20
21
22
23
24
25

무엇을 보게 되나

JSON 한 줄이 펼쳐집니다. 값이 아니라 어떤 항목을 남겼는지를 보세요.

`collected` → `picked` → `drafted` → `published` — 갈때기입니다. 네 숫자를 나란히 놓으면 단계마다 얼마나 좁혀졌는지가 보여요. `picked` 와 `drafted` 가 다르면 취재에서 빠진 건이 있다는 뜻이고, `drafted` 와 `published` 가 다르면 검수에서 떨어진 겁니다. 섹션 8과 섹션 9에서 "조용히 줄어드는"고 했던 자리가 여기서 숫자로 드러납니다.

`hours` 는 그 실행에서 쓴 시간 창입니다. 발행 건수가 자꾸 적으면 이 값을 늘려 볼 근거가 되고, 바꾼 날짜를 기록으로 되짚을 수 있습니다.

`by_source` — 발행된 다섯 건이 어느 소스에서 왔는지입니다. 잠시 뒤 성적표의 재료가 되고,

어떤 소스가 몇 주째 한 건도 못 올리는지를 이 숫자로만 알 수 있습니다.

`log` — 실행 중에 찍힌 줄을 그대로 담았습니다. 응답하지 않은 소스, 본문이 짧아 빠진 기사가

여기 들어 있어서 나중에 "오늘 왜 이 기사가 안 실렸지?"에 답할 수 있습니다.

한 줄이 이 정도를 말해 준다는 게 요점입니다. 관측에 필요한 건 도구가 아니라 무엇을 남길지 정하는 일이에요.

이 파일은 어디에 생기나

`store/metrics.jsonl` 은 코드가 돌아간 그 컴퓨터에 생깁니다. 그래서 어디서 돌렸느냐에 따라

파일이 다른 곳에 쌓여요.

어디서 돌렸나	파일이 생기는 곳	오래 남나
내 노트북	노트북의 <code>store/metrics.jsonl</code>	남는다. 돌릴 때마다 줄이 늘어난다
GitHub Actions	그 회차용 임시 서버	사라진다. 실행이 끝나면 서버째 없어진다

그러니까 Actions 로 돌린 결과는 **내 노트북 파일에 한 줄도 추가되지 않습니다.**

둘은 서로 다른 컴퓨터예요. 섹션 11에서 `upload-artifact` 를 붙여 뒀으니 실행 페이지에서 내려받을 수는 있지만, 그건 **그 회차 한 줄짜리 파일**입니다. 회차끼리 이어 붙지 않아요.

누적하려면 실행이 끝날 때 그 파일을 저장소에 되커밋해야 합니다. 워크플로 마지막에 붙입니다.

```
permissions:
  contents: write# 저장소에 쓰려면 필요하다

jobs:
  publish:
    steps:
      # ... 앞은 그대로 ...
      -name: 기록 커밋
      if: ${!inputs.dry_run}
      run:|
        git config user.name "newsroom-bot"
        git config user.email "bot@users.noreply.github.com"
        git add store/
        git diff --cached --quiet || (git commit -m "metrics: $(date +%F)" && git pu
sh)
```

이렇게 해 두면 `git pull` 할 때마다 그날 기록이 내 노트북으로 따라옵니다.

캐시나 artifact 와 달리 커밋은 지워지지 않고, 덤으로 **언제 무엇이 발행됐는지**가 **git 이력에 남아** 나중에 되짚을 수 있습니다.

두 가지만 알아 두세요. 로컬에서도 돌리고 있다면 같은 파일에 양쪽 기록이 쌓여 **충돌이 날 수 있습니다.** 한 줄씩 덧붙이는 파일이라 충돌이 나면 두 쪽을 위아래로 이어 붙이면 됩니다. 그리고 `store/` 안에 API 응답 원문 같은 걸 같이 남기면 **그것까지 저장소에 공개되니**, 커밋할 파일은 지표 정도로 제한하세요.

깔때기 관측

여러 단계를 거쳐 좁혀지는 파이프라인에서 각 단계의 입출력 개수를 나란히 기록하는 것입니다.

우리 경우 수집 → 예선 → 본선 → 발행의 네 숫자가 그것입니다.

이게 없으면 고장이 어느 단계에서 났는지 알 수 없어요. "오늘 뉴스레터가 부실하다"는 관찰만으로는 소스가 죽은 것인지 예선이 뻑뻑한 것인지 구분이 안 됩니다. 깔때기를 보면 숫자가 갑자기 줄어든 자리가 범인이예요.

각 단계가 끝날 때 개수를 세어 같은 줄에 기록하면 됩니다. 단계별 비율을 함께 보는 게 중요해요. 절대 개수는 그날의 뉴스 양에 따라 출렁이지만 비율은 안정적이라, 예선 통과율이 늘 25% 안팎이다가 갑자기 60%가 되면 후보가 줄었거나 기준이 느슨해진 겁니다.

다만 깔때기는 어디가 문제인지는 알려 주지만 무엇이 문제인지는 알려 주지 않습니다. 예선에서 숫자가 확 줄었다는 걸 알아도 후보가 나빠서인지 기준이 빡빡해서인지는 구분되지 않아요. 그래서 개수와 함께 탈락 사유를 남겨야 합니다. 수집 노드가 `dead` 를, 취재 노드가 '본문 부족'을 로그에 남긴 이유가 그겁니다. 그리고 이 숫자들은 며칠은 쌓여야 의미가 생깁니다.

소스별 성적표

깔때기는 파이프라인 전체를 봅니다. 그런데 섹션 4의 관문과 등급은 소스별이었죠. 쌓인 기록을 소스 기준으로 다시 세면 성적표가 됩니다. 새 도구가 필요하지 않아요.

`by_source` 를 더하기만 하면 됩니다.

실행 되었습니다.

실습: 쌓인 기록으로 소스 성적표 만들기 (키 불필요)

실행

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15

16
17
18
19
20
21
22
23

무엇을 보게 되나

소스마다 한 줄씩 발행 기여 건수가 찍히고, 그 아래 최근 실행의 깔때기가 나옵니다. 세 가지를 보세요.

기여가 0인 소스가 보일 겁니다. 수집은 되는데 발행까지 한 번도 못 올라간 곳이에요. 며칠 치로는 우연일 수 있지만 몇 주가 지나도 0이면 그 소스는 매일 요청과 예선 토큰을 쓰면서 아무것도 기여하지 않는 겁니다. 빨 소스를 숫자가 지목해 주는 것, 이게 성적표의 쓸모입니다.

기여가 한쪽으로 몰린 것도 보일 수 있습니다. 한 소스가 매일 다섯 자리 중 서너 개를 가져간다면 나머지 소스가 후보에조차 못 드는 상태일 수 있어요. 그럴 때 매체별 상한을 코드로 거는 것이 다음 수인데, 섹션 7의 더 해보기에 그 방법이 있습니다.

깔때기 줄에서는 [수집 → 선별](#) 의 낙폭과 [취재 → 발행](#) 의 낙폭을 따로 보세요. 앞쪽이 크면 정상이고(그게 선별의 일입니다), 뒤쪽이 크면 본문 추출이나 검수에서 조용히 빠지고 있다는 뜻입니다.

값은 여러분의 소스 목록과 돌린 횟수에 따라 다릅니다. 중요한 건 이 표가 새 도구 없이 **이미 남긴 기록만으로** 나왔다는 점이에요.

이 결정을 검증하는 방법

확인할 것 — 정한 소스 목록과 선별 구조가 제 몫을 하고 있다.

아래 신호로 확인합니다.

- **소스별 발행 기여** — [by_source](#) 를 누적해 소스마다 몇 건이 실렸는지 셉니다. 몇 주 내 내 0이면 그 소스는 비용만 쓰고 있으므로 뺍니다.

- **깔때기 단계별 비율** — 수집 → 예선 → 본선 → 발행의 네 숫자를 매 실행 기록하고 비율로 봅니다. 한 단계의 비율이 갑자기 변하면 그 단계 또는 바로 앞 단계가 바뀐 것입니다.
- **발행 건수의 분포** — 매일 몇 건이 나갔는지 셉니다. 0건인 날도 함께 셉니다. 0건이나 1~2건이 잦으면 시간 창이나 소스 수가 부족하다는 뜻입니다.

세 지표 모두 이미 출력되고 있는 값을 파일에 남기기만 하면 됩니다. 방금 `run()` 이 그 일을 하고, 성적표 셀이 그 누적 기록을 읽습니다. 소스별 기여는 밖에서 썰 수 없고 우리 기록으로만 알 수 있는 지표예요.

틀렸을 때의 증상 — 지표를 안 남겼을 때의 증상은 결정을 되돌릴 수 없게 되는 것입니다. 몇 달 뒤 "이 소스를 왜 넣었지?"라는 질문에 기록이 없으면 답은 "그때 괜찮아 보여서"뿐이고 뽕 근거도 없어서 쓸모없는 소스가 목록에 영원히 남습니다. 기록이 없으면 결정이 쌓이기만 하고 정리되지 않습니다.

객관식 퀴즈

Q. 어떤 소스가 섹션 4의 관문은 전부 통과했고 본문 길어도 넉넉한데, 성적표에서는 발행 기여가 몇 주째 0이라고 합시다. 이 조합이 뜻하는 것으로 가장 적절한 것은?

- 1. 본문 추출이 실제로는 실패하고 있으므로 G1 측정 코드에 버그가 있다
- 2. 기술적으로는 쓸 수 있는 소스지만 우리 독자 기준에서 중요한 기사를 내놓지 않는다는 뜻이다
- 3. 예선 프롬프트가 영어 기사를 차별하고 있으므로 프롬프트를 고쳐야 한다
- 4. 표본이 적어서 생긴 우연이므로 무시해도 된다 **제출**

• •

1. 본문 추출이 실제로는 실패하고 있으므로 G1 측정 코드에 버그가 있다

• •

2. 기술적으로는 쓸 수 있는 소스지만 우리 독자 기준에서 중요한 기사를 내놓지 않는다는 뜻이다

• •

3. 예선 프롬프트가 영어 기사를 차별하고 있으므로 프롬프트를 고쳐야 한다

• •

4. 표본이 적어서 생긴 우연이므로 무시해도 된다

더 해보기 — 내 파이프라인의 깔때기 만들기

1. 실행마다 깔때기 네 숫자를 jsonl 파일에 append 하는 코드를 열 줄로 작성한다

- 접근 힌트: `json.dumps(row, ensure_ascii=False)` 로 한 줄씩 만들고 `open(path, 'a')` 로 덧붙인다. `run_id` 로 날짜를 넣어 두면 나중에 날짜별로 묶을 수 있다
- 성공하면: 관측을 시작하는 데 필요한 코드가 정말 열 줄이면 된다는 것을 확인할 수 있다. 도구가 없어서 관측을 안 한 것이 아니었음을 알게 된다

2. 쌓인 기록을 읽어 단계별 통과율 추이를 그래프로 그린다

- 접근 힌트: pandas 로 jsonl 을 읽고 (`pd.read_json(path, lines=True)`) `shortlist/collected` 비율을 날짜순으로 그린다
- 성공하면: 며칠치가 쌓이면 '정상 범위'가 보이기 시작한다. 그 범위를 벗어나는 날을 자동으로 알리는 것이 다음 단계다



관측과 지표에 관한 참고 자료

- [JSON Lines 형식 소개](#)

Chapter13

구조와 내용 분리하기

지금까지 세운 구조 — 사다리, 관문, 예선·본선, 코드 강제, 그래프, 깔때기 — 중에 AI 뉴스에만 해당하는 건 하나도 없습니다. 채용 공고를 모으든 논문을 모으든 같아요.

그런데 코드를 열어 보면 곳곳에 AI 뉴스가 박혀 있습니다. 프롬프트에 "국내 개발팀"이 적혀 있고, "이번 주 우리가 일하는 방식이 바뀔 만한가"가 적혀 있고, 토픽 이름에 "모델·API"가 있죠.

구조는 일반적인데 내용이 특수한 상태입니다. 이 둘을 분리하지 않으면 다른 분야로 옮길 때 파이썬 파일을 전부 열어 문자열을 찾아 고쳐야 해요. 어디에 무엇이 박혀 있는지 기억하는 사람만 할 수 있는 일이 되고, 그 사람이 떠나면 아무도 못 옮깁니다.

선택지

정해야 할 것 — 독자·기준·토픽처럼 '분야마다 달라지는 내용'을 어디에 둘 것인가?

네 가지가 있고, 아래로 갈수록 고치기는 쉬워지지만 만들 것이 많아집니다.

① 프롬프트 문자열 안에 직접

`SYS = "당신은 국내 개발팀을 위한..."` 처럼 코드 안에 그대로 씁니다. 가장 빠르고, 프롬프트를 읽으면 전체 맥락이 한눈에 보입니다. 프로토타입 단계에서는 이것이 맞습니다. 대신 같은

내용이 여러 프롬프트에 흩어져 한 곳만 고치는 사고가 납니다. 그리고 고칠 때마다 파이썬 파일을 여는 것은 비개발자에게는 불가능한 일입니다.

② 파이썬 상수 모듈로 모으기

`config.py` 에 `AUDIENCE = "..."` 같은 상수를 모아 둡니다. 한 곳에 모이므로 흩어짐 문제가 해결되고, 코드 자동완성과 타입 검사가 됩니다. 대신 여전히 파이썬 파일입니다. 편집자나 기획자가 고치려면 따옴표와 들여쓰기를 다뤄야 하고, 실수하면 프로그램이 아예 안 됩니다.

③ 설정 파일(YAML·TOML)로 분리 ← 우리가 고른 것

`audience.yaml` 같은 파일에 독자·기준·토픽·톤을 적고 코드가 읽어 씁니다. 프로그래밍을 모르는 사람도 고칠 수 있습니다. 주석으로 작성 요령을 적어 둘 수 있고, 무엇이 설정이고 무엇이 코드인지 경계가 파일로 드러납니다. 버전 관리에 남아 '왜 기준이 바뀌었는지'도 추적됩니다. 대신 검증이 없으면 오타가 런타임에 터집니다. 그리고 파일이 커지면 결국 아무도 전체를 읽지 않게 됩니다.

④ 데이터베이스 + 관리자 화면

설정을 DB에 넣고 웹 화면에서 고치게 합니다. 여러 뉴스레터를 동시에 운영하거나, 고치는 사람이 여럿일 때 필요해집니다. 대신 만들어야 할 것이 갑자기 많아집니다. 화면, 권한, 이력 관리까지. 뉴스레터 하나를 운영하는 단계에서는 과합니다.

우리는 설정 파일(YAML·TOML)로 분리를 골랐습니다. 기준은 누가 이것을 고칠 것인가입니다. 독자 정의나 중요도 기준은 개발자가 아니라 편집 방향을 정하는 사람이 고쳐야 합니다. 그 사람에게 파이썬 파일을 열게 하면 실제로는 아무도 안 고치게 됩니다. ④는 고치는 사람이 여럿이 될 때 넘어갈 곳이지 지금 필요한 것이 아닙니다.

③의 약점인 '검증 없음'은 스키마 검사로 메울 수 있습니다. 설정을 읽을 때 `pydantic` 같은 것으로 한 번 검증하면, 오타는 프로그램이 시작하자마자 잡힙니다. 새벽 실행 중간에 터지는 것보다 훨씬 낫습니다.

조건이 달라지면 다른 선택지가 맞는 답이 됩니다. 무엇을 포기했는지 알고 고르면 됩니다.

설정 파일 예시

지금까지 코드에 박아 둔 `CRITERIA` 문자열을 파일로 옮기면 이런 모양이 됩니다.

```
독자:
누구: "AI를 실제 제품에 붙이는 국내 개발팀 - 개발자와 기획자"
이미_아는_것: "LLM API 사용법, 프롬프트 기본, RAG 개념"

# 본선 프롬프트에 그대로 들어간다. 위에 있을수록 우선.
# 기사 하나를 보고 예/아니오로 답할 수 있는 질문이어야 한다.
중요도_기준:
- "이번 주 우리가 일하는 방식이 바뀔 만한가"
- "지금 쓰는 도구·API의 가격·한도·정책이 실제로 변했나"
```

```
# 추상어 ("홍보성") 말고 구체적인 표현으로.  
버릴_것:  
- "발표 예정·로드맵만 있고 지금 쓸 수 없는 것"  
- "MOU·업무협약·투자유치·수상 등 당사자 홍보성 소식"  
  
토픽:  
- 이름: "모델·API"  
데스크지침: "가격·컨텍스트 길이·요청 한도 변화를 반드시 숫자로 적을 것"  
- 이름: "정책·규제"  
데스크지침: "규제 대상과 시행 시점을 반드시, 국내 적용 여부도"
```

실행 되었습니다.

준비: YAML 읽기

실행

1

실행 되었습니다.

실습: 설정 파일을 읽어 프롬프트에 끼우기 (키 불필요)

실행

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

무엇을 보게 되나

`audience.yaml` 의 내용이 문단 하나로 조립되어 찍힙니다. 그게 그대로 프롬프트로 들어갑니다.

기존 코드는 한 줄도 고치지 않았습니다. 섹션 7의 `ask_picks` 안을 보면

`sys = (f"{CRITERIA}...")` 였죠. 함수가 불릴 때마다 그 시점의 `CRITERIA` 를 읽으니, 위 셀에서 값을 바꿔 두면 다음 `run()` 부터 새 기준이 들어갑니다. 섹션 8의 `SYS` (취재 프롬프트)도 같은 이유로 그대로 갈아 끼워집니다.

`graph.py` 로 옮길 때는 이 로딩 코드를 `CRITERIA` 를 정의하던 자리에 대신 넣으면 됩니다. 문자열 상수 자리에 `build_criteria(CFG)` 가 들어가는 것뿐이에요.

`requirements.txt` 에 `pyyaml` 한 줄 추가하는 것도 잊지 마세요.

파일이 없을 때 만들어 두는 `if not path.exists()` 는 교안용 편의입니다.

실제 프로젝트에서는 파일이 없으면 **바로 멈추는 편이** 낫습니다 —
설정을 못 읽은 채로 기본값으로 돌면 그게 조용한 실패니까요.

이 파일에서 배울 것은 YAML 문법이 아니라 주석입니다. 세 개가 붙어 있습니다.

기사 하나를 보고 예/아니오로 답할 수 있는 질문이어야 한다 — 이것이
중요도 기준을 쓰는 규칙입니다. "좋은 기사"처럼 판정할 수 없는 문장을 적으면
모델이 매번 다르게 해석하므로 형태를 제약해 둔 것입니다.

추상어("홍보성") 말고 구체적인 표현으로 — 섹션 1의 아이스브레이킹에서
'번거로움을 구체적인 장면으로 적으라'고 했던 것과 같은 이야기입니다.
"홍보성 기사를 버려라"는 모델마다 다르게 해석하지만
"MOU·업무협약·투자유치·수상"은 거의 틀리지 않습니다.

위에 있을수록 우선 — 기준끼리 충돌할 때 어떻게 할지를 미리 정한 것입니다.

**설정 파일을 잘 만든다는 것은 값을 옮겨 놓는 것이 아니라, 그 값을 어떻게 채워야 하는지를
파일 안에 적어 두는 것입니다.** 이 주석들이 없으면 다음 사람은 빈칸을 앞에 두고
무엇을 적어야 할지 모릅니다.

설정과 코드의 경계

설정은 같은 프로그램을 다른 상황에 쓰기 위해 바꾸는 값이고, 코드는 상황이 바뀌어도 그
대로인 절차입니다. 우리 경우 '중요도 기준 네 문장'은 설정이고, '예선에서 묶음별로 뽑고
본선에서 비교한다'는 코드예요.

이 경계가 흐려지면 재사용이 불가능해집니다. 다른 분야로 옮길 때 고쳐야 할 곳이 코드 전
체에 흩어져 있으면 옮기는 것보다 새로 만드는 게 빨라져요. 그러면 같은 구조를 여러 번 다
시 만들게 되고, 각각이 조금씩 다르게 남아 갑니다.

경계를 정하는 질문은 두 개입니다. "분야가 바뀌면 이 값이 바뀌는가"와 "이걸 고칠 사람이
개발자인가". 첫 질문에 예이면 설정 후보이고, 두 번째에 아니오이면 설정 파일로 빼야 해
요.

`audience.yaml` 은 둘 다 해당하고, `settings.yaml` 의 임계값들(`BATCH` , `TARGET`)은 첫 질문에
만 해당해서 별도 파일에 두었습니다.

그렇다고 모든 걸 설정으로 빼면 아무것도 이해할 수 없게 됩니다. 설정 항목이 많아질수록
프로그램의 실제 동작을 알려면 코드와 설정을 함께 읽어야 하고, 조합의 수가 많아 테스트
도 어려워져요. 그리고 설정에는 기본값이 반드시 있어야 합니다. 비어 있을 때 프로그램이
죽으면 설정으로 뺀 게 오히려 취약점이 되니까요. 판단 기준은 '바꿀 수 있는가'가 아니라
'실제로 바뀐 적이 있는가'에 가깝습니다.

객관식 퀴즈

Q. `audience.yaml`의 '버릴_것' 항목에 "홍보성 기사" 대신 "MOU·업무협약·투자유치·수상 등 당사자 홍보성 소식"처럼 적는 이유로 가장 적절한 것은?

- 1. 문장이 길수록 모델이 더 중요하게 취급하기 때문이다
 - 2. '홍보성'은 판정 기준이 사람마다 다른 추상어여서 모델이 매번 다르게 해석하지만, 구체적인 유형 목록은 해석의 여지가 적기 때문이다
 - 3. 한국어 모델은 명사 나열을 더 잘 이해하기 때문이다
 - 4. YAML 파일에는 추상적인 단어를 쓸 수 없다는 형식 제약이 있기 때문이다
- 제출
- •
 - 1. 문장이 길수록 모델이 더 중요하게 취급하기 때문이다
 - •
 - 2. '홍보성'은 판정 기준이 사람마다 다른 추상어여서 모델이 매번 다르게 해석하지만, 구체적인 유형 목록은 해석의 여지가 적기 때문이다
 - •
 - 3. 한국어 모델은 명사 나열을 더 잘 이해하기 때문이다
 - •
 - 4. YAML 파일에는 추상적인 단어를 쓸 수 없다는 형식 제약이 있기 때문이다

주관식 퀴즈

Q. 섹션 1의 아이스브레이킹에서 여러분이 적은 '내 분야의 소식과 그 번거로움'을 다시 꺼내 보세요. 그 번거로움을 `audience.yaml`의 `중요도\_기준`이나 `버릴\_것` 항목으로 옮긴다면 어떤 문장이 될까요? '기사 하나를 보고 예/아니오로 답할 수 있는 질문'이라는 규칙과 '추상어 말고 구체적으로'라는 규칙을 지켜서 적어 보세요.

0/300다음

더 해보기 — 내 분야의 설정 파일 만들기

1. 여러분 분야로 `audience.yaml`을 처음부터 새로 써 본다

- 접근 힌트: 독자·중요도_기준(34개)·버릴_것(45개)·토픽(3~5개)을 채운다. 각 항목에 '어떻게 채워야 하는지'를 주석으로 함께 적는다
- 성공하면: 구조는 그대로 두고 내용만 바꿔도 완전히 다른 뉴스레터가 된다는 것을 확인할 수 있다. 그리고 주석을 쓰는 일이 값을 쓰는 일보다 어렵다는 것도 알게 된다

2. 설정을 pydantic 모델로 검증해 오타가 시작 시점에 잡히게 만든다

- 접근 힌트: `class Audience(BaseModel)` 로 필드를 정의하고 `Audience(**yaml.safe_load(...))` 로 읽는다. 필수 항목을 빼먹었을 때 어떤 오류가 나는지 확인한다
- 성공하면: 설정 파일의 약점(런타임에 터짐)이 스무 줄로 메워진다는 것을 경험할 수 있다

Chapter14

만든 것

섹션 2에서 빈 노드 다섯 개를 세웠고, 섹션 5·7·9·11·12에서 하나씩 채웠습니다.

지금 그래프를 돌리면 다섯 줄이 전부 0이 아닌 숫자로 찍히고 마지막에 디스코드로 알림이 갑니다.

섹션 11에서 그걸 매일 아침 자동으로 돌게 만들었고요.

이제 무엇을 결정했는지 되짚어 봅시다.

배운 것

한 단계씩 세우는 순서. 빈 노드 다섯 개를 먼저 만들고 섹션마다 하나씩 채웠습니다.

매 섹션 끝에 전체를 한 번 돌렸으니 어디까지 왔는지가 늘 로그에 보였고,

단계 사이에 무엇을 주고받을지도 맨 처음 한 곳에 적었습니다.

선택지를 먼저 늘어놓는 순서. 섹션마다 답을 먼저 말하지 않고 후보를 전부 늘어놓은 뒤 하나를 골랐습니다. 만드는 순서, 수집 경로, 선별 구조, 제약의 위치, 출력 구조, 검수 방식, 못 채우는 날의 처리, 발행 경로, 설정의 위치 — 모두 고른 것보다 버린 것을 아는 것이 목적이었습니다.

① **수집 — 관문과 등급은 다른 것을 쟀다.** 합산으로 상쇄되면 안 되는 것(관문)과 우열을 가리는 것(등급)을 나누었고, 관문을 통과했다고 좋은 소스가 아니라는 것을 확인했습니다.

② **선별 — 개수가 고정된 문제는 채점이 아니라 정렬 문제다.** 후보를 한 화면에 놓고 견주어야 하고, 후보가 많으면 묶음 사이의 비교를 포기하는 대가를 치릅니다. 그리고 프롬프트에 적은 부탁이 늘 지켜지지 않는다는 것도 여기서 봤습니다.

③ **요약 — 출력의 구조가 검증 가능성을 정한다.** 취재 결과를 세 칸으로 나눈 것은 형식의 문제가 아니라 나중에 무엇을 자동으로 검사할 수 있을지를 정하는 일이었습니다. 한 덩어리로 받았다면 그 구분 자체가 불가능했습니다.

④ **검수** — '무엇을 무엇과 대조할 것인가'의 문제다. 숫자 대조는 값싸지만 번역과 단위 환산을

오탐했습니다. 검사가 틀린 것이 아니라 대조 방식이 문제에 안 맞았던 것입니다.

빠대 — 펼칠 수 있는 단계와 없는 단계가 있다. 기준은 성능이 아니라

"이 단계는 다른 항목을 봐야 답할 수 있는가"였습니다. 그리고 건수가 모자라는 날은 되돌아가지 않고 적은 대로 발행했습니다. 단순한 규칙 하나가 재시도 설계 전체를 대신합니다.

⑤ **발행** — 되돌릴 수 없는 곳은 다르게 다룬다. 보내기 전에 무엇을 보낼지 보는

`dry_run` 을 먼저 만들었고, 그 단계에는 언어 모델을 넣지 않았습니다.

그리고 실제로 받아 봤습니다 — 매일 아침 알림이 오는 것까지가 만든 것입니다.

측정. JSON 한 줄과 성적표 한 장으로 내린 결정 대부분을 검증했습니다.

숫자는 우리가 못 본 것을 지목해 줍니다.

다루지 않은 것

평가셋과 회귀 검사. 프롬프트를 고칠 때 "좋아진 것 같다"로 판단했습니다. 제대로 하려면 기사 30~50건에 사람이 정답을 붙인 평가셋 위에서 점수를 재야 고쳤더니 다른 데가 나빠지는 것을 잡을 수 있습니다.

사람이 개입하는 지점. 우리 파이프라인은 승인 없이 발행합니다. 발행 전에 사람이 한 번 보는 구조로 바꾸려면 그래프를 멈췄다 재개하는 기능이 필요합니다.

비용과 지연의 최적화. 지금은 모든 단계에 같은 모델을 씁니다. 예선처럼 판단이 단순한 곳은 더 싼 모델로 내리는 배분이 가능합니다.

소스 확장과 스크래핑. 섹션 3의 사다리에서 5칸은 "쓰지 않는다"로 닫았습니다.

1~4칸이 없는 분야를 다루려면 그 칸이 필요하고, 그때는 robots.txt 확인과 구조 변경 감지가 통째로 따라옵니다.

주관식 퀴즈

Q. [회고] 섹션 1에서 여러분이 적었던 '매일 챙겨 보는 분야와 그 번거로움'을 다시 읽어 보세요. 그리고 섹션 13에서 그것을 `audience.yaml` 항목으로 옮겨 보았습니다. 다섯 단계 중 여러분 분야로 옮길 때 가장 크게 손 봐야 할 단계는 어디이고 그 이유는 무엇인가요? 그대로 써도 되는 단계는 어디인가요?